

## ▼ Data Visualisation Portfolio — COM7021

- **Name:** Muhammad Sameer
- **Student ID:** 24169956

**Module:** Data Visualisation | **Code:** COM7021

**Dataset:** European Bakery Sales (2000–2005)

**Tools:** Python · Pandas · Plotly

### ▼ 1. Install & Import Libraries

```
import pandas as pd
import numpy as np
import plotly.express as px
import plotly.graph_objects as go
from plotly.subplots import make_subplots
import json
import plotly.io as pio
from IPython.display import display, HTML
import warnings
warnings.filterwarnings('ignore')

from google.colab import files
```

### ▼ 2. Upload & Load Dataset

```
bakery_dataset = pd.read_excel('/content/Data Visualisation - COM7021 - [4566] Bakery- supporting document.xlsx')

print(f"Shape: {bakery_dataset.shape[0]} rows x {bakery_dataset.shape[1]} columns")
print(f"\nColumns: {bakery_dataset.columns.tolist()}")
bakery_dataset.head()
```

Shape: 1001 rows x 7 columns

Columns: ['Date', 'City', 'Confectionary', 'Units Sold', 'Revenue(£)', 'Cost(£)', 'Profit(£)']

|   | Date       | City   | Confectionary | Units Sold | Revenue(£) | Cost(£) | Profit(£) |  |
|---|------------|--------|---------------|------------|------------|---------|-----------|--|
| 0 | 2002-11-11 | London | Biscuit       | 1118.0     | 5590.0     | 2459.6  | 3130.4    |  |
| 1 | 2002-07-05 | London | Biscuit       | 708.0      | 3540.0     | 1557.6  | 1982.4    |  |
| 2 | 2001-10-31 | London | Biscuit       | 1269.0     | 6345.0     | 2791.8  | 3553.2    |  |
| 3 | 2004-09-13 | London | Biscuit       | 1631.0     | 8155.0     | 3588.2  | 4566.8    |  |
| 4 | 2004-03-10 | London | Biscuit       | 2240.0     | 11200.0    | 4928.0  | 6272.0    |  |

Next steps: [Generate code with bakery\\_dataset](#) [New interactive sheet](#)

### ▼ 3. Data Cleaning & Preprocessing

```
print(bakery_dataset.isnull().sum())
print(f"\nTotal missing: {bakery_dataset.isnull().sum().sum()}")
```

```
Date      0
City      0
Confectionary  0
Units Sold  5
Revenue(£)  9
Cost(£)    9
Profit(£)  3
dtype: int64
```

Total missing: 26

```
print("Confectionery values BEFORE cleaning:")
print(bakery_dataset['Confectionary'].value_counts())
```

```
Confectionery values BEFORE cleaning:
Confectionary
Caramel          345
Chocolate Chunk  130
Biscuit Nut      125
Biscuit          124
Plain           112
Caramel nut       82
Caramel Nut       67
Chocolate Chunk   16
Name: count, dtype: int64
```

```
bakery_dataset['Confectionary'] = bakery_dataset['Confectionary'].str.strip()

typo_map = {
    'Chocolate Chunk': 'Chocolate Chunk',
    'Caramel nut': 'Caramel Nut',
}
bakery_dataset['Confectionary'] = bakery_dataset['Confectionary'].replace(typo_map)

print("\nConfectionery values AFTER cleaning:")
print(bakery_dataset['Confectionary'].value_counts())
```

```
Confectionery values AFTER cleaning:
Confectionary
Caramel          345
Caramel Nut      149
Chocolate Chunk  146
Biscuit Nut      125
Biscuit          124
Plain           112
Name: count, dtype: int64
```

```
for col in ['Units Sold', 'Revenue(£)', 'Cost(£)', 'Profit(£)']:
    bakery_dataset[col] = bakery_dataset.groupby('City')[col].transform(
        lambda x: x.fillna(x.median())
    )

print("Missing values AFTER cleaning:")
print(bakery_dataset.isnull().sum())
```

```
Missing values AFTER cleaning:
Date          0
City          0
Confectionary 0
Units Sold    0
Revenue(£)    0
Cost(£)       0
Profit(£)     0
dtype: int64
```

```
bakery_dataset['Year'] = bakery_dataset['Date'].dt.year
bakery_dataset['Month'] = bakery_dataset['Date'].dt.to_period('M').dt.to_timestamp()
bakery_dataset['YearStr'] = bakery_dataset['Year'].astype(str)

print("Date range:", bakery_dataset['Date'].min().date(), "to", bakery_dataset['Date'].max().date())
print("Cities:", bakery_dataset['City'].unique().tolist())
print("Confectioneries:", bakery_dataset['Confectionary'].unique().tolist())
```

```
Date range: 2000-01-02 to 2005-12-28
Cities: ['London', 'Paris', 'Bonn', 'Seville', 'Napoli']
Confectioneries: ['Biscuit', 'Biscuit Nut', 'Chocolate Chunk', 'Caramel Nut', 'Caramel', 'Plain']
```

```
print(bakery_dataset.shape)
```

```
(1001, 10)
```

## 4. Exploratory Data Analysis (EDA)

```
bakery_dataset[['Units Sold', 'Revenue(£)', 'Cost(£)', 'Profit(£)']].describe().round(2)
```

|              | Units Sold | Revenue(£) | Cost(£) | Profit(£) |
|--------------|------------|------------|---------|-----------|
| <b>count</b> | 1001.00    | 1001.00    | 1001.0  | 1001.00   |
| <b>mean</b>  | 1632.63    | 6842.14    | 2817.0  | 4011.70   |
| <b>std</b>   | 874.27     | 4678.85    | 2065.0  | 2644.21   |
| <b>min</b>   | 200.00     | 200.00     | 40.0    | 160.00    |
| <b>25%</b>   | 941.00     | 3010.00    | 1228.5  | 1873.60   |
| <b>50%</b>   | 1531.00    | 5984.00    | 2459.6  | 3460.80   |
| <b>75%</b>   | 2296.00    | 9495.00    | 3908.0  | 5445.00   |
| <b>max</b>   | 4493.00    | 23988.00   | 10994.5 | 13479.00  |

```
print("=== Total Profit by City ===")
city_totals = bakery_dataset.groupby('City')['Profit(£)'].sum().sort_values(ascending=False)
display(city_totals.apply(lambda x: f'£{x:,.2f}'))
```

```
=== Total Profit by City ===
```

|                | Profit(£)   |
|----------------|-------------|
| <b>City</b>    |             |
| <b>Paris</b>   | £936,914.20 |
| <b>Napoli</b>  | £913,678.70 |
| <b>Seville</b> | £746,995.00 |
| <b>Bonn</b>    | £723,059.50 |
| <b>London</b>  | £695,061.12 |

```
dtype: object
```

```
print("\n=== Total Profit by Confectionery ===")
conf_totals = bakery_dataset.groupby('Confectionery')['Profit(£)'].sum().sort_values(ascending=False)
display(conf_totals.apply(lambda x: f'£{x:,.2f}'))
```

```
=== Total Profit by Confectionery ===
```

|                        | Profit(£)     |
|------------------------|---------------|
| <b>Confectionery</b>   |               |
| <b>Caramel</b>         | £1,643,008.38 |
| <b>Caramel Nut</b>     | £712,104.75   |
| <b>Biscuit</b>         | £605,680.80   |
| <b>Biscuit Nut</b>     | £515,297.50   |
| <b>Chocolate Chunk</b> | £391,583.50   |
| <b>Plain</b>           | £148,033.60   |

```
dtype: object
```

```
display(bakery_dataset.groupby(['City', 'Confectionery'])['Profit(£)'].sum().unstack().round(0))
```

| Confectionary  | Biscuit  | Biscuit Nut | Caramel  | Caramel Nut | Chocolate Chunk | Plain   |  |
|----------------|----------|-------------|----------|-------------|-----------------|---------|--|
| City           |          |             |          |             |                 |         |  |
| <b>Bonn</b>    | 127434.0 | 108095.0    | 224928.0 | 152692.0    | 79065.0         | 30846.0 |  |
| <b>London</b>  | 46130.0  | 61928.0     | 451486.0 | 81890.0     | 53627.0         | NaN     |  |
| <b>Napoli</b>  | 154112.0 | 109220.0    | 343494.0 | 166628.0    | 102382.0        | 37843.0 |  |
| <b>Paris</b>   | 154060.0 | 141045.0    | 336582.0 | 179290.0    | 86324.0         | 39613.0 |  |
| <b>Seville</b> | 123945.0 | 95010.0     | 286518.0 | 131606.0    | 70186.0         | 39731.0 |  |

```
# — Consistent colour palette used across all charts —
CITY_COLORS = {
    'London': '#2E86AB',
    'Paris': '#A23B72',
    'Bonn': '#F18F01',
    'Seville': '#C73E1D',
    'Napoli': '#3B1F2B',
}
CONF_COLORS = px.colors.qualitative.Bold
```

## 5. Chart 1 — European Bakery City Performance Overview

```
# =====
# Chart 1 — European Bakery City Performance Overview
# (City + Year checkbox filters + live info box)
# =====

city_coords = {
    'London': {'lat': 51.5074, 'lon': -0.1278},
    'Paris': {'lat': 48.8566, 'lon': 2.3522},
    'Bonn': {'lat': 50.7374, 'lon': 7.0982},
    'Seville': {'lat': 37.3891, 'lon': -5.9845},
    'Napoli': {'lat': 40.8518, 'lon': 14.2681},
}

yearly_summary_df = bakery_dataset.groupby(['Year', 'City'], as_index=False).agg(
    Revenue=('Revenue(£)', 'sum'),
    Profit=('Profit(£)', 'sum'),
    Units=('Units Sold', 'sum'),
    Transactions=('Date', 'count'),
)
yearly_summary_df['lat'] = yearly_summary_df['City'].map(lambda c: city_coords[c]['lat'])
yearly_summary_df['lon'] = yearly_summary_df['City'].map(lambda c: city_coords[c]['lon'])
yearly_summary_df['Margin'] = (yearly_summary_df['Profit'] / yearly_summary_df['Revenue'] * 100).round(2)

city_alltime = bakery_dataset.groupby('City', as_index=False).agg(
    Revenue=('Revenue(£)', 'sum'),
    Profit=('Profit(£)', 'sum'),
    Units=('Units Sold', 'sum'),
    Transactions=('Date', 'count'),
)
city_alltime['lat'] = city_alltime['City'].map(lambda c: city_coords[c]['lat'])
city_alltime['lon'] = city_alltime['City'].map(lambda c: city_coords[c]['lon'])
city_alltime['Margin'] = (city_alltime['Profit'] / city_alltime['Revenue'] * 100).round(2)

global_max_revenue = max(yearly_summary_df['Revenue'].max(), city_alltime['Revenue'].max())

_all_years = sorted(yearly_summary_df['Year'].unique().tolist())
_all_cities = sorted(yearly_summary_df['City'].unique().tolist())

fig1 = make_subplots(
    rows=2, cols=2,
    column_widths=[0.29, 0.55],
    row_heights=[0.56, 0.44],
    specs=[
        [{'type': 'scattergeo', 'rowspan': 2}, {'type': 'bar'}],
        [None, {'type': 'bar'}],
    ],
    subplot_titles=('', 'Total Revenue by City', 'Profit Margin (%) by City'),
)
```

```

for city in _all_cities:
    row = city_alltime[city_alltime['City'] == city].iloc[0]
    clr = CITY_COLORS.get(city, '#888')
    sz = (row['Revenue'] / global_max_revenue) * 50 + 14
    textposition = 'bottom center' if city == 'Paris' else 'top center'

    fig1.add_trace(go.Scattergeo(
        lat=[row['lat']], lon=[row['lon']],
        mode='markers+text', name=city,
        text=[f"<b>{city}</b><br>£{row['Revenue']/1e6:.2f}M"],
        textposition=textposition,
        textfont=dict(size=12, color=clr, family='Arial Black'),
        marker=dict(size=sz, color=clr, opacity=0.88, line=dict(width=2.5, color='white')),
        hovertemplate=(
            f"<b>{city}</b><br>Year: All Years<br>"
            f"Revenue: £{row['Revenue']:.0f}<br>Profit: £{row['Profit']:.0f}<br>"
            f"Margin: {row['Margin']:.1f}%<br>Units: {row['Units']:.0f}<extra></extra>"
        ),
        showlegend=True, legendgroup=city, visible=True,
    ), row=1, col=1)

    fig1.add_trace(go.Bar(
        y=[city], x=[row['Revenue']], orientation='h',
        marker_color=clr,
        text=[f"£{row['Revenue']/1e6:.2f}M"],
        textposition='outside', textfont=dict(size=11, family='Arial'),
        hovertemplate=f"<b>{y}</b><br>Year: All Years<br>Revenue: £{x:.0f}<extra></extra>",
        name=city, legendgroup=city, showlegend=False, visible=True,
    ), row=1, col=2)

    fig1.add_trace(go.Bar(
        y=[city], x=[row['Margin']], orientation='h',
        marker=dict(color=clr, opacity=0.78),
        text=[f"{row['Margin']:.1f}%"],
        textposition='outside', textfont=dict(size=11, family='Arial'),
        hovertemplate=f"<b>{y}</b><br>Year: All Years<br>Margin: {x:.1f}%<extra></extra>",
        name=city, legendgroup=city, showlegend=False, visible=True,
    ), row=2, col=2)

fig1.update_geos(
    scope='europe',
    showland=True, landcolor='#f5f5f0',
    showocean=True, oceancolor='#d4e8f0',
    showcountries=True, countrycolor='#c8c8c8', countrywidth=1,
    showcoastlines=True, coastlinecolor='#b0b0b0',
    showlakes=True, lakecolor='#d4e8f0',
    center={'lat': 47, 'lon': 3},
    projection_scale=3.0,
)
fig1.update_layout(
    title=dict(
        text='<b>European Bakery – City Performance Overview (All Years)</b>',
        font=dict(size=18),
    ),
    font=dict(family='Arial', size=13),
    template='plotly_white',
    height=560,
    width=1200,
    margin=dict(t=80, b=20, l=20, r=70),
    legend=dict(
        title='City', x=0.5, y=-0.15, xanchor='center', yanchor='top',
        bgcolor='rgba(255,255,255,0.88)',
        bordercolor='ddd', borderwidth=1,
        orientation='h',
    ),
)
fig1.update_xaxes(tickprefix='£', tickformat=',.0f', showgrid=True, gridcolor='eee', row=1, col=2)
fig1.update_xaxes(ticksuffix='%', showgrid=True, gridcolor='eee', row=2, col=2)
fig1.update_yaxes(showgrid=False)
fig1.update_annotations(font_color='#444')

yearly_js = json.dumps(
    yearly_summary_df[['Year', 'City', 'Revenue', 'Profit', 'Units', 'Transactions', 'lat', 'lon', 'Margin']]
    .to_dict(orient='records')
)

```

```

cities_js = json.dumps(_all_cities)
years_js = json.dumps([int(y) for y in _all_years])
colors_js = json.dumps(CITY_COLORS)
glob_max_js = json.dumps(float(global_max_revenue))

post_script_1 = f"""
(function() {{

    var gd = document.getElementById('fig1-overview');
    var yearlyData = {yearly_js};
    var allCities = {cities_js};
    var allYears = {years_js};
    var cityColors = {colors_js};
    var globalMaxRev = {glob_max_js};

    // — state —————
    var selCities = allCities.slice();
    var selYears = allYears.slice();

    var traceSnap = null;
    function ensureSnap() {{
        if (!traceSnap && gd.data && gd.data.length > 0) {{
            traceSnap = JSON.parse(JSON.stringify(gd.data));
        }}
    }}

    // Checkbox dropdown helper —————

    function makeDropdown(idBase, labelText, items, isNumeric) {{
        var wrap = document.createElement('div');
        wrap.style.cssText = 'position:relative;display:inline-flex;flex-direction:column;';

        var capLabel = document.createElement('span');
        capLabel.textContent = labelText;
        capLabel.style.cssText = 'font-size:10px;color:#888;font-weight:600;letter-spacing:.04em;margin-bottom:3px;';
        wrap.appendChild(capLabel);

        var btn = document.createElement('button');
        btn.id = idBase + '-btn';
        btn.style.cssText = [
            'padding:6px 10px 6px 12px',
            'border:1px solid #d4d4d4',
            'border-radius:7px',
            'font-size:12px',
            'font-family:Arial,sans-serif',
            'background:#fff',
            'color:#333',
            'cursor:pointer',
            'outline:none',
            'min-width:130px',
            'display:flex',
            'justify-content:space-between',
            'align-items:center',
            'gap:6px',
            'white-space:nowrap',
        ].join(';');
        btn.innerHTML = '<span id="' + idBase + '-lbl">All</span><span style="font-size:10px;color:#999;">▼</span>';
        wrap.appendChild(btn);

        var menu = document.createElement('div');
        menu.id = idBase + '-menu';
        menu.style.cssText = [
            'position:absolute',
            'top:100%',
            'left:0',
            'z-index:9999',
            'background:#fff',
            'border:1px solid #d4d4d4',
            'border-radius:9px',
            'box-shadow:0 6px 20px rgba(0,0,0,.13)',
            'padding:6px 0',
            'min-width:155px',
            'max-height:230px',
            'overflow-y:auto',
            'display:none',
            'margin-top:3px',
        ].join(';');
    }}

```

```

J.join( );

// "Select All" row at top of menu
var allRow = document.createElement('div');
allRow.style.cssText = 'padding:7px 12px;font-size:12px;cursor:pointer;display:flex;align-items:center;gap:8px;color:#333';
var allCb = document.createElement('input');
allCb.type = 'checkbox';
allCb.id = idBase + '-all';
allCb.checked = true;
allCb.style.cssText = 'cursor:pointer;accent-color:#2563eb;';
allRow.appendChild(allCb);
allRow.appendChild(document.createTextNode('Select All'));
menu.appendChild(allRow);

items.forEach(function(v) {{
  var itemRow = document.createElement('div');
  itemRow.style.cssText = 'padding:6px 12px;font-size:12px;cursor:pointer;display:flex;align-items:center;gap:8px;color:#333';
  itemRow.addEventListener('mouseenter', function() {{ itemRow.style.background = '#f5f7ff'; }});
  itemRow.addEventListener('mouseleave', function() {{ itemRow.style.background = ''; }});

  var cb = document.createElement('input');
  cb.type = 'checkbox';
  cb.value = String(v);
  cb.className = idBase + '-cb';
  cb.checked = true;
  cb.style.cssText = 'cursor:pointer;accent-color:#2563eb;';

  itemRow.appendChild(cb);
  itemRow.appendChild(document.createTextNode(String(v)));
  menu.appendChild(itemRow);
}});

wrap.appendChild(menu);

// toggle open/close
btn.addEventListener('click', function(e) {{
  e.stopPropagation();
  document.querySelectorAll('[id$="-menu"]').forEach(function(m) {{
    if (m !== menu) m.style.display = 'none';
  }});
  menu.style.display = menu.style.display === 'block' ? 'none' : 'block';
}});
document.addEventListener('click', function() {{ menu.style.display = 'none'; }});
menu.addEventListener('click', function(e) {{ e.stopPropagation(); }});

// "Select All" toggle
allCb.addEventListener('change', function() {{
  document.querySelectorAll('.' + idBase + '-cb').forEach(function(cb) {{
    cb.checked = allCb.checked;
  }});
  syncState(idBase, isNumeric);
  renderAll();
}});

// individual checkbox
menu.querySelectorAll('.' + idBase + '-cb').forEach(function(cb) {{
  cb.addEventListener('change', function() {{
    var noneChecked = !Array.from(document.querySelectorAll('.' + idBase + '-cb')).some(function(c) {{ return c.checked; }});
    var allChecked = Array.from(document.querySelectorAll('.' + idBase + '-cb')).every(function(c) {{ return c.checked; }});
    allCb.checked = allChecked;
    // if user just unchecked the last one, snap it back (require at least 1)
    if (noneChecked) {{ cb.checked = true; allCb.checked = false; }}
    syncState(idBase, isNumeric);
    renderAll();
  }});
}});

return wrap;
}}

// — sync selection arrays from checkboxes —————
function syncState(idBase, isNumeric) {{
  var sel = Array.from(document.querySelectorAll('.' + idBase + '-cb'))
    .filter(function(cb) {{ return cb.checked; }})
    .map(function(cb) {{ return isNumeric ? Number(cb.value) : cb.value; }});

  if (idBase === 'f1-city') selCities = sel;

```

```

else
    selYears = sel;

var total = isNumeric ? allYears.length : allCities.length;
var lbl = document.getElementById(idBase + '-lbl');
if (!lbl) return;
if (sel.length === total) lbl.textContent = 'All';
else
    lbl.textContent = sel.length + ' selected';
}}

// — aggregate stats per city for current year selection —————
function getStats() {{
    var stats = {};
    allCities.forEach(function(city) {{
        var rev = 0, profit = 0, units = 0, txns = 0, coord = null;
        yearlyData.forEach(function(d) {{
            if (d.City === city && selYears.indexOf(d.Year) !== -1) {{
                rev += d.Revenue;
                profit += d.Profit;
                units += d.Units;
                txns += d.Transactions;
                if (!coord) coord = {{ lat: d.lat, lon: d.lon }};
            }}
        }});
        stats[city] = {{
            rev: rev, profit: profit, units: units, txns: txns,
            margin: rev > 0 ? (profit / rev * 100) : 0,
            coord: coord,
        }};
    }});
    return stats;
}}

// — info box update —————
function updateInfoBox(stats) {{
    var box = document.getElementById('fig1-info-box');
    if (!box) return;

    var totalRev = 0, totalProfit = 0, totalUnits = 0;
    selCities.forEach(function(city) {{
        if (stats[city] && stats[city].rev > 0) {{
            totalRev += stats[city].rev;
            totalProfit += stats[city].profit;
            totalUnits += stats[city].units;
        }}
    }});
    var combinedMargin = totalRev > 0 ? (totalProfit / totalRev * 100) : 0;

    var cityLine = selCities.length === allCities.length
        ? '<em>All cities</em>'
        : selCities.join(', ');
    var yrLine = selYears.length === allYears.length
        ? '<em>2000 - 2005</em>'
        : selYears.slice().sort(function(a,b){{ return a-b; }}).join(', ');

    function card(label, val, isBlue) {{
        var color = isBlue ? '#1d4ed8' : '#1f2937';
        var size = isBlue ? '15px' : '13px';
        var weight = isBlue ? '800' : '600';
        return '<div style="flex:1;min-width:110px;">' +
            '<div style="font-size:10px;color:#9ca3af;font-weight:600;letter-spacing:.05em;margin-bottom:2px;">' + label +
            '<div style="font-size:' + size + ';color:' + color + ';font-weight:' + weight + ';line-height:1.3;">' + val +
            '</div>';
    }}

    box.innerHTML =
        '<div style="font-size:11px;font-weight:700;letter-spacing:.06em;color:#6b7280;margin-bottom:9px;border-bottom:1px'
        '<div style="display:flex;gap:18px;flex-wrap:wrap;">' +
            card('CITIES', cityLine, false) +
            card('YEARS', yrLine, false) +
            card('TOTAL REVENUE', '£' + totalRev.toLocaleString('en-GB', {{maximumFractionDigits:0}}), true) +
            card('TOTAL PROFIT', '£' + totalProfit.toLocaleString('en-GB', {{maximumFractionDigits:0}}), true) +
            card('AVG MARGIN', combinedMargin.toFixed(1) + '%', false) +
            card('UNITS SOLD', totalUnits.toLocaleString('en-GB', {{maximumFractionDigits:0}}), false) +
            '</div>';
}}

```



```
// — main render via Plotly.react —————
function renderAll() {{
  ensureSnap();
  if (!traceSnap) return;

  var stats = getStats();
  var yrLabel = selYears.length === allYears.length ? 'All Years' :
    selYears.length === 1 ? String(selYears[0]) :
    selYears.length + ' years selected';

  // deep clone the original trace structure (preserves subplot axis refs)
  var newTraces = JSON.parse(JSON.stringify(traceSnap));

  allCities.forEach(function(city, i) {{
    var geoIdx = i * 3;
    var revIdx = i * 3 + 1;
    var marIdx = i * 3 + 2;

    var s = stats[city];
    var inSel = selCities.indexOf(city) !== -1 && s && s.rev > 0;
    var clr = cityColors[city] || '#888';
    var sz = inSel ? (s.rev / globalMaxRev) * 50 + 14 : 14;
    var textposition = city === 'Paris' ? 'bottom center' : 'top center';

    // — geo (scattergeo) trace —————
    newTraces[geoIdx].visible = inSel;
    newTraces[geoIdx].lat = [s && s.coord ? s.coord.lat : 0];
    newTraces[geoIdx].lon = [s && s.coord ? s.coord.lon : 0];
    newTraces[geoIdx].text = inSel
      ? ['<b>' + city + '</b><br>£' + (s.rev / 1e6).toFixed(2) + 'M']
      : [''];
    newTraces[geoIdx].textposition = textposition;
    newTraces[geoIdx].marker = {{
      size: sz, color: clr, opacity: 0.88,
      line: {{ width: 2.5, color: 'white' }}
    }};
    newTraces[geoIdx].hovertemplate = inSel
      ? '<b>' + city + '</b><br>Year: ' + yrLabel +
        '<br>Revenue: £' + s.rev.toLocaleString('en-GB', {{maximumFractionDigits:0}}) +
        '<br>Profit: £' + s.profit.toLocaleString('en-GB', {{maximumFractionDigits:0}}) +
        '<br>Margin: ' + s.margin.toFixed(1) + '%' +
        '<br>Units: ' + s.units.toLocaleString('en-GB', {{maximumFractionDigits:0}}) +
        '<extra></extra>'
      : '<extra></extra>';

    // — revenue bar trace —————
    newTraces[revIdx].visible = inSel;
    newTraces[revIdx].x = [inSel ? s.rev : 0];
    newTraces[revIdx].text = [inSel ? '£' + (s.rev / 1e6).toFixed(2) + 'M' : ''];
    newTraces[revIdx].hovertemplate = '<b>%{y}</b><br>Year: ' + yrLabel + '<br>Revenue: £%{x:,.0f}<extra></extra>';

    // — margin bar trace —————
    newTraces[marIdx].visible = inSel;
    newTraces[marIdx].x = [inSel ? s.margin : 0];
    newTraces[marIdx].text = [inSel ? s.margin.toFixed(1) + '%' : ''];
    newTraces[marIdx].hovertemplate = '<b>%{y}</b><br>Year: ' + yrLabel + '<br>Margin: %{x:.1f}<extra></extra>';
  }});

  Plotly.react(gd, newTraces, gd.layout);

  // update chart title to reflect current year context
  Plotly.relayout(gd, {{
    'title.text': '<b>European Bakery – City Performance Overview (' + yrLabel + ')</b>'
  }});

  updateInfoBox(stats);
}}

// Build and inject the filter panel —————

var panel = document.createElement('div');
panel.style.cssText = [
  'display:flex',
  'align-items:flex-end',
  'flex-wrap:wrap',
  'gap:12px',
  'padding:12px 16px',
];
```

```

        'margin-bottom:8px',
        'background:#fff',
        'border:1px solid #e5e7eb',
        'border-radius:10px',
        'font-family:Arial,sans-serif',
        'box-sizing:border-box',
        'width:100%',
    ].join(';');

    panel.appendChild(makeDropdown('f1-city', 'CITY', allCities, false));
    panel.appendChild(makeDropdown('f1-year', 'YEAR', allYears, true));

    var rst = document.createElement('button');
    rst.textContent = '↺ Reset';
    rst.style.cssText = 'padding:6px 14px;background:#f3f4f6;color:#374151;border:1px solid #d1d5db;border-radius:7px;font-size';
    rst.addEventListener('mouseover', function() {{ rst.style.background = '#e5e7eb'; }});
    rst.addEventListener('mouseout', function() {{ rst.style.background = '#f3f4f6'; }});
    rst.addEventListener('click', function() {{
        ['f1-city', 'f1-year'].forEach(function(id) {{
            document.querySelectorAll('.' + id + '-cb, #' + id + '-all').forEach(function(cb) {{
                cb.checked = true;
            }});
            var lbl = document.getElementById(id + '-lbl');
            if (lbl) lbl.textContent = 'All';
        }});
        selCities = allCities.slice();
        selYears = allYears.slice();
        renderAll();
    }});
    panel.appendChild(rst);
    gd.parentNode.insertBefore(panel, gd);

    // — info box —————

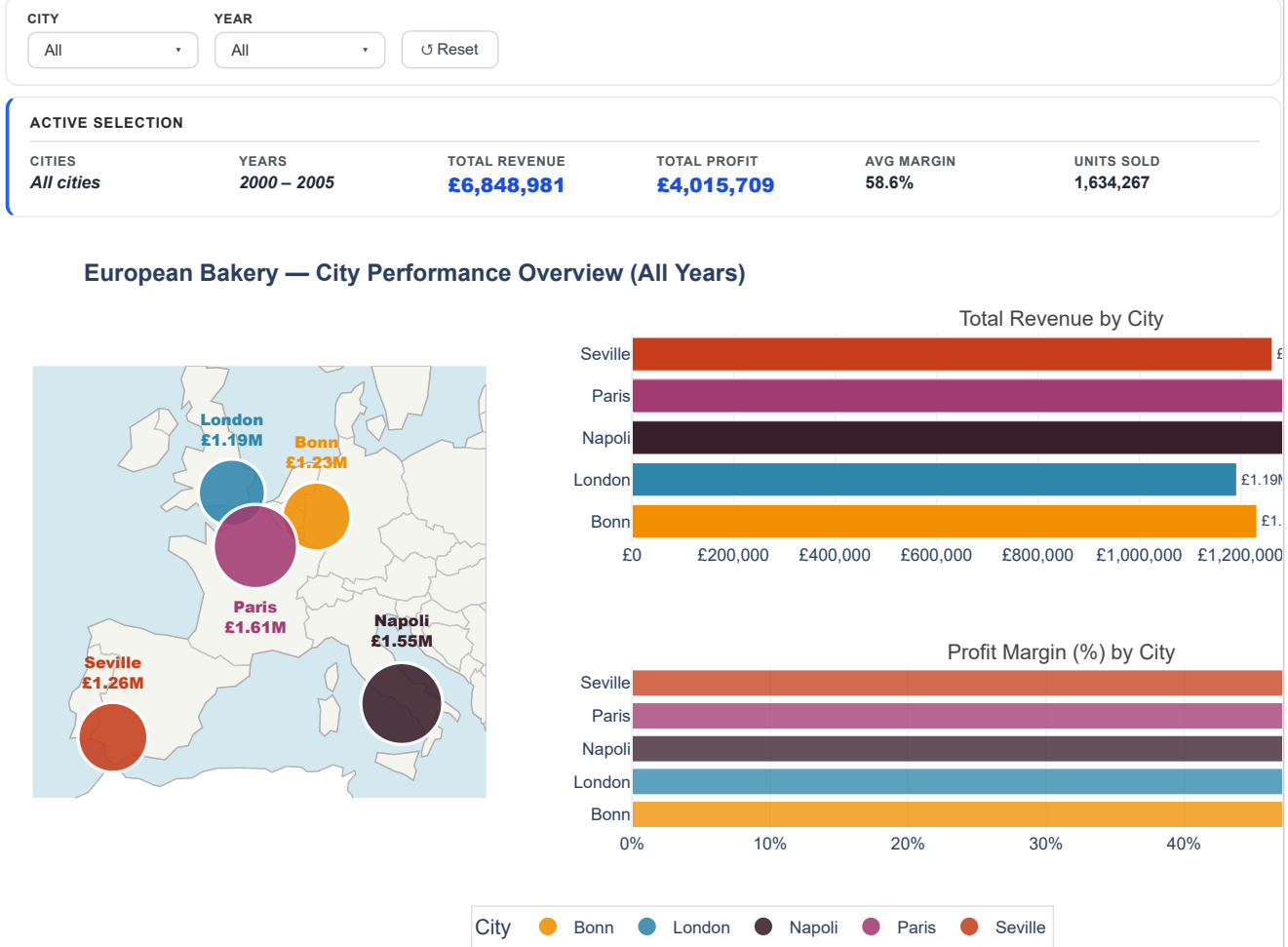
    var infoBox = document.createElement('div');
    infoBox.id = 'fig1-info-box';
    infoBox.style.cssText = [
        'padding:13px 16px',
        'margin-bottom:8px',
        'background:#f9fafb',
        'border:1px solid #e5e7eb',
        'border-left:3px solid #2563eb',
        'border-radius:10px',
        'font-family:Arial,sans-serif',
        'box-sizing:border-box',
        'width:100%',
    ].join(';');
    gd.parentNode.insertBefore(infoBox, gd);

    // initial snapshot + render after Plotly finishes drawing
    gd.on('plotly_afterplot', function() {{
        if (!traceSnap) {{
            traceSnap = JSON.parse(JSON.stringify(gd.data));
            updateInfoBox(getStats()); // populate info box on first load
        }}
    }});

    }}());
    ""

    ch1_html = pio.to_html(
        fig1, full_html=False, include_plotlyjs='cdn',
        config=dict(displaylogo=False, responsive=True),
        div_id='fig1-overview', post_script=post_script_1,
    )
    display(HTML(ch1_html))

```



## 6. Chart 2 — Revenue Sunburst (City → Confectionery)

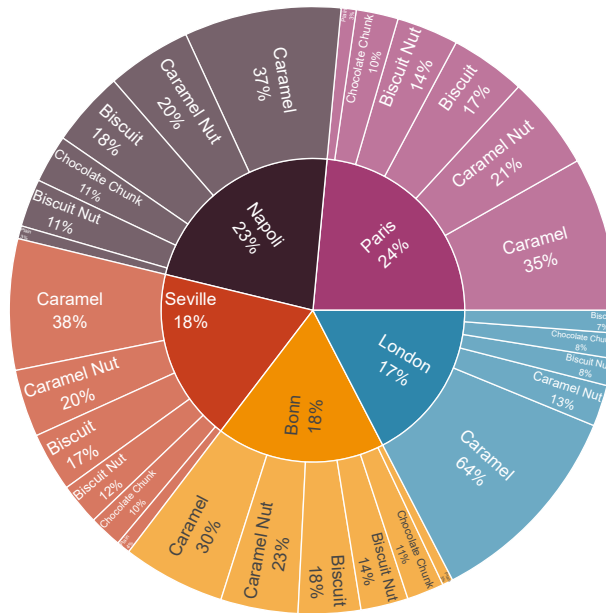
```
# =====
# Chart 2 – Revenue Sunburst (City → Confectionery)
# =====

fig2 = px.sunburst(
    bakery_dataset,
    path=['City', 'Confectionery'],
    values='Revenue(£)',
    color='City',
    color_discrete_map=CITY_COLORS,
    title='<b>Revenue Sunburst: City → Confectionery</b>',
    template='plotly_white',
)
fig2.update_traces(
    hovertemplate=(
        '<b>{label}</b><br>'
        'Revenue: £{value:,.0f}<br>'
        '%{percentParent:.1%} of %{parent}<br>'
        '%{percentRoot:.1%} of total'
        '<extra></extra>'
    ),
    textinfo='label+percent parent',
    insidetextorientation='radial',
)
fig2.update_layout(
    title_font_size=18,
    font=dict(family='Arial', size=13),
    margin=dict(t=50, b=20, l=20, r=20),
    height=520,
)
```

```
ps2 = None

ch2_html = pio.to_html(
    fig2, full_html=False, include_plotlyjs='cdn',
    config=dict(displaylogo=False), div_id='fig2-sunburst',
)
display(HTML(ch2_html))
```

### Revenue Sunburst: City → Confectionery



## 7. Chart 3 — Annual Cost Heatmap by Confectionery per City

```
# =====
# Chart 3 – Annual Cost Heatmap by Confectionery per City
# =====

if 'Year' not in bakery_dataset.columns:
    bakery_dataset['Year'] = bakery_dataset['Date'].dt.year

# aggregate: total cost per Year, City, Confectionery
heat_agg = (
    bakery_dataset
    .groupby(['Year', 'City', 'Confectionery'], as_index=False)['Cost(£)']
    .sum()
)

all_years = sorted(heat_agg['Year'].unique().tolist())
all_cities = sorted(heat_agg['City'].unique().tolist())
all_confs = sorted(heat_agg['Confectionery'].unique().tolist())

# build initial z matrix – all cities, all years, confs sorted by total desc
def build_z(data, confs, years):
    z, txt = [], []
    for conf in confs:
        row_z, row_t = [], []
        for yr in years:
            val = float(data.loc[
                (data['Confectionery'] == conf) & (data['Year'] == yr), 'Cost(£)'
            ].sum())
            row_z.append(val if val > 0 else None)
            row_t.append(f'£{val:,.0f}' if val > 0 else '')
        z.append(row_z)
        txt.append(row_t)
    return z, txt
```

```

conf_totals = heat_agg.groupby('Confectionary')['Cost(£)'].sum().sort_values(ascending=False)
init_confs = conf_totals.index.tolist()

z_init, txt_init = build_z(heat_agg, init_confs, all_years)

fig_heat = go.Figure(go.Heatmap(
    z=z_init,
    x=[str(y) for y in all_years],
    y=init_confs,
    colorscale='Viridis',
    colorbar=dict(
        title=dict(text='Total Cost (£)', side='right'),
        tickprefix='£',
        tickformat=',.0f',
        len=0.85,
    ),
    hovertemplate='<b>{y}</b><br>Year: {x}<br>Total Cost: £{z:,.2f}<extra></extra>',
    text=txt_init,
    texttemplate='%{text}',
    textfont=dict(size=12, color='white'),
    zsmooth=False,
))

fig_heat.update_layout(
    title=dict(
        text='<b>Annual Cost Heatmap – Confectionery by City & Year</b>',
        font=dict(size=17, family='Arial'),
        x=0,
        xanchor='left',
    ),
    font=dict(family='Arial', size=12),
    template='plotly_white',
    height=460,
    margin=dict(t=55, b=50, l=170, r=20),
    xaxis=dict(title='Year', type='category', tickfont=dict(size=14)),
    yaxis=dict(title='Confectionery Item', autorange='reversed', tickfont=dict(size=14)),
)

# — pass data to JavaScript —————
heat_data_js = json.dumps(heat_agg.to_dict(orient='records'))
all_years_js = json.dumps(all_years)
all_cities_js = json.dumps(all_cities)
all_confs_js = json.dumps(all_confs)

post_script = f"""
(function() {{

    var gd      = document.getElementById('fig-heatmap');
    var rawData = {heat_data_js};
    var allYears = {all_years_js};
    var allCities = {all_cities_js};
    var allConfs = {all_confs_js};

    // — selection state (mirrors "all selected" at start) —————
    var selCities = allCities.slice();
    var selConfs = allConfs.slice();
    var selYears = allYears.slice();

    // Dropdown with checkboxes —————

    function makeDropdown(idBase, labelText, items, isNumeric) {{

        var wrap = document.createElement('div');
        wrap.style.cssText = 'position:relative;display:inline-flex;flex-direction:column;';

        var capLabel = document.createElement('span');
        capLabel.textContent = labelText;
        capLabel.style.cssText = 'font-size:10px;color:#888;font-weight:600;letter-spacing:.04em;margin-bottom:3px;';
        wrap.appendChild(capLabel);

        var btn = document.createElement('button');
        btn.id = idBase + '-btn';
        btn.style.cssText = 'padding:6px 10px 6px 12px;border:1px solid #d4d4d4;border-radius:7px;font-size:12px;font-family:
        btn.innerHTML = '<span id="' + idBase + '-lbl">All</span><span style="font-size:10px;color:#999;">•</span>';

```

```

wrap.appendChild(btn);

var menu = document.createElement('div');
menu.id = idBase + '-menu';
menu.style.cssText = 'position:absolute;top:100%;left:0;z-index:9999;background:#fff;border:1px solid #d4d4d4;border-

// "Select All" row

var allRow = document.createElement('div');
allRow.style.cssText = 'padding:7px 12px;font-size:12px;cursor:pointer;display:flex;align-items:center;gap:8px;color:
var allCb = document.createElement('input');
allCb.type = 'checkbox';
allCb.id = idBase + '-all';
allCb.checked = true;
allCb.style.cssText = 'cursor:pointer;accent-color:#2563eb;';
allRow.appendChild(allCb);
allRow.appendChild(document.createTextNode('Select All'));
menu.appendChild(allRow);

items.forEach(function(v) {{
  var row = document.createElement('div');
  row.style.cssText = 'padding:6px 12px;font-size:12px;cursor:pointer;display:flex;align-items:center;gap:8px;color
  row.addEventListener('mouseenter', function() {{ row.style.background = '#f5f7ff'; }});
  row.addEventListener('mouseleave', function() {{ row.style.background = ''; }});

  var cb = document.createElement('input');
  cb.type = 'checkbox';
  cb.value = String(v);
  cb.className = idBase + '-cb';
  cb.checked = true;
  cb.style.cssText = 'cursor:pointer;accent-color:#2563eb;';

  row.appendChild(cb);
  row.appendChild(document.createTextNode(String(v)));
  menu.appendChild(row);
}});

wrap.appendChild(menu);

// — toggle menu open/close —————

btn.addEventListener('click', function(e) {{
  e.stopPropagation();
  document.querySelectorAll('[id$="-menu"]').forEach(function(m) {{
    if (m !== menu) m.style.display = 'none';
  }});
  menu.style.display = menu.style.display === 'block' ? 'none' : 'block';
}});

document.addEventListener('click', function() {{ menu.style.display = 'none'; }});
menu.addEventListener('click', function(e) {{ e.stopPropagation(); }});

// — "Select All" toggle —————

allCb.addEventListener('change', function() {{
  document.querySelectorAll('.' + idBase + '-cb').forEach(function(cb) {{
    cb.checked = allCb.checked;
  }});
  syncState(idBase, isNumeric);
  renderAll();
}});

// — individual cb —————

menu.querySelectorAll('.' + idBase + '-cb').forEach(function(cb) {{
  cb.addEventListener('change', function() {{
    var cbs = document.querySelectorAll('.' + idBase + '-cb');
    var allChecked = Array.from(cbs).every(function(c) {{ return c.checked; }});
    allCb.checked = allChecked;
    syncState(idBase, isNumeric);
    renderAll();
  }});
}});

return wrap;
}}

```

```
// — sync checkbox state into selection arrays —————

function syncState(idBase, isNumeric) {{
  var sel = Array.from(document.querySelectorAll('.' + idBase + '-cb'))
    .filter(function(cb) {{ return cb.checked; }})
    .map(function(cb) {{ return isNumeric ? Number(cb.value) : cb.value; }});

  if (idBase === 'f-city') selCities = sel;
  else if (idBase === 'f-conf') selConfs = sel;
  else selYears = sel;

  // update button label

  var allItems = isNumeric ? allYears : (idBase === 'f-city' ? allCities : allConfs);
  var lbl = document.getElementById(idBase + '-lbl');
  if (!lbl) return;
  if (sel.length === 0) lbl.textContent = 'None';
  else if (sel.length === allItems.length) lbl.textContent = 'All';
  else lbl.textContent = sel.length + ' selected';
}}

// — build z matrix, sorted by total cost desc —————

function buildMatrix(confs, years, cities) {{
  // compute per-conf total so we can sort rows by it
  var totals = {};
  confs.forEach(function(conf) {{
    var t = 0;
    rawData.forEach(function(d) {{
      if (d.Confectionary === conf &&
          cities.indexOf(d.City) !== -1 &&
          years.indexOf(d.Year) !== -1) t += d['Cost(£)'];
    }});
    totals[conf] = t;
  }});

  var sorted = confs.slice().sort(function(a, b) {{ return totals[b] - totals[a]; }});

  var z = [], txt = [], yLabels = [];
  sorted.forEach(function(conf) {{
    var zRow = [], tRow = [];
    years.forEach(function(yr) {{
      var v = 0;
      rawData.forEach(function(d) {{
        if (d.Confectionary === conf &&
            d.Year === yr &&
            cities.indexOf(d.City) !== -1) v += d['Cost(£)'];
      }});
      zRow.push(v > 0 ? v : null);
      tRow.push(v > 0 ? '£' + v.toLocaleString('en-GB', {{minimumFractionDigits:0, maximumFractionDigits:0}}) : '');
    }});
    z.push(zRow);
    txt.push(tRow);
    yLabels.push(conf);
  }});

  return {{ z: z, txt: txt, y: yLabels }};
}}

// — update legend/info box —————

function updateLegend() {{
  var box = document.getElementById('heatmap-info-box');
  if (!box) return;

  var grandTotal = 0;
  rawData.forEach(function(d) {{
    if (selCities.indexOf(d.City) !== -1 &&
        selConfs.indexOf(d.Confectionary) !== -1 &&
        selYears.indexOf(d.Year) !== -1) {{
      grandTotal += d['Cost(£)'];
    }}
  }});

  var cityLine = selCities.length === allCities.length
```

```

    ? '<em>All cities</em>'
    : (selCities.length === 0 ? '<em style="color:#e53e3e;">None selected</em>' : selCities.join(', '));

var conflate = selConfs.length === allConfs.length
? '<em>All items</em>'
: (selConfs.length === 0 ? '<em style="color:#e53e3e;">None selected</em>' : selConfs.join(', '));

var yrLine = selYears.length === allYears.length
? '<em>2000 - 2005</em>'
: (selYears.length === 0 ? '<em style="color:#e53e3e;">None selected</em>'
  : selYears.slice().sort(function(a,b){return a-b;}).join(', '));

box.innerHTML =
  '<div style="font-size:11px;font-weight:700;letter-spacing:.06em;color:#6b7280;margin-bottom:9px;">ACTIVE SELECTI
  '<div style="margin-bottom:7px;">' +
    '<div style="font-size:10px;color:#9ca3af;font-weight:600;letter-spacing:.05em;">CITIES</div>' +
    '<div style="font-size:12px;color:#1f2937;line-height:1.45;">' + cityLine + '</div>' +
  '</div>' +
  '<div style="margin-bottom:7px;">' +
    '<div style="font-size:10px;color:#9ca3af;font-weight:600;letter-spacing:.05em;">ITEMS</div>' +
    '<div style="font-size:12px;color:#1f2937;line-height:1.45;">' + conflate + '</div>' +
  '</div>' +
  '<div style="margin-bottom:10px;">' +
    '<div style="font-size:10px;color:#9ca3af;font-weight:600;letter-spacing:.05em;">YEARS</div>' +
    '<div style="font-size:12px;color:#1f2937;line-height:1.45;">' + yrLine + '</div>' +
  '</div>' +
  '<div style="border-top:1px solid #e5e7eb;padding-top:9px;">' +
    '<div style="font-size:10px;color:#9ca3af;font-weight:600;letter-spacing:.05em;margin-bottom:2px;">COMBINED TOT
    '<div style="font-size:16px;font-weight:800;color:#1d4ed8;">£' +
      grandTotal.toLocaleString('en-GB', {{minimumFractionDigits:2, maximumFractionDigits:2}}) +
    '</div>' +
  '</div>';
}}

// — render / react —————

function renderAll() {{
  var years = selYears.slice().sort(function(a, b) {{ return a - b; }});
  var mat = buildMatrix(selConfs, years, selCities);

  Plotly.react(gd, [{{
    type: 'heatmap',
    z: mat.z,
    x: years.map(String),
    y: mat.y,
    colorscale: 'Viridis',
    colorbar: {{
      title: {{ text: 'Total Cost (£)', side: 'right' }},
      tickprefix: '£',
      tickformat: ',.0f',
      len: 0.85,
    }},
    hovertemplate: '<b>{{y}}</b><br>Year: {{x}}<br>Total Cost: £{{z:.2f}}<extra></extra>',
    text: mat.txt,
    texttemplate: '{{text}}',
    textfont: {{ size: 12, color: 'white' }}, // Increased font size for heatmap text
    zsmooth: false,
  }}, gd.layout);

  updateLegend();
}}

// Build the filter panel

var panel = document.createElement('div');
panel.style.cssText = [
  'display:flex',
  'align-items:flex-end',
  'flex-wrap:wrap',
  'gap:12px',
  'padding:12px 16px',
  'margin-bottom:8px',
  'background:#fff',
  'border:1px solid #e5e7eb',
  'border-radius:10px',
  'font-family:Arial,sans-serif',

```



```

        'box-sizing:border-box',
        'width:100%',
    ].join(';');

    panel.appendChild(makeDropdown('f-city', 'CITY', allCities, false));
    panel.appendChild(makeDropdown('f-conf', 'CONFECTIONERY', allConfs, false));
    panel.appendChild(makeDropdown('f-year', 'YEAR', allYears, true));

    var rst = document.createElement('button');
    rst.textContent = '↺ Reset';
    rst.style.cssText = 'padding:6px 14px;background:#f3f4f6;color:#374151;border:1px solid #d1d5db;border-radius:7px;font-si
    rst.addEventListener('mouseover', function() {{ rst.style.background = '#e5e7eb'; }});
    rst.addEventListener('mouseout', function() {{ rst.style.background = '#f3f4f6'; }});
    rst.addEventListener('click', function() {{
        ['f-city', 'f-conf', 'f-year'].forEach(function(id) {{
            document.querySelectorAll('.' + id + '-cb, #' + id + '-all').forEach(function(cb) {{
                cb.checked = true;
            }});
            var lbl = document.getElementById(id + '-lbl');
            if (lbl) lbl.textContent = 'All';
        }});
        selCities = allCities.slice();
        selConfs = allConfs.slice();
        selYears = allYears.slice();
        renderAll();
    }});
    panel.appendChild(rst);

    gd.parentNode.insertBefore(panel, gd);

    // Info / legend box – sits right below the filter bar, above chart

    var infoBox = document.createElement('div');
    infoBox.id = 'heatmap-info-box';
    infoBox.style.cssText = [
        'padding:13px 16px',
        'margin-bottom:8px',
        'background:#f9fafb',
        'border:1px solid #e5e7eb',
        'border-left:3px solid #2563eb',
        'border-radius:10px',
        'font-family:Arial,sans-serif',
        'box-sizing:border-box',
        'width:100%',
        'display:flex',
        'gap:28px',
        'flex-wrap:wrap',
        'align-items:flex-start',
    ].join(';');

    gd.parentNode.insertBefore(infoBox, gd);

    // initial render

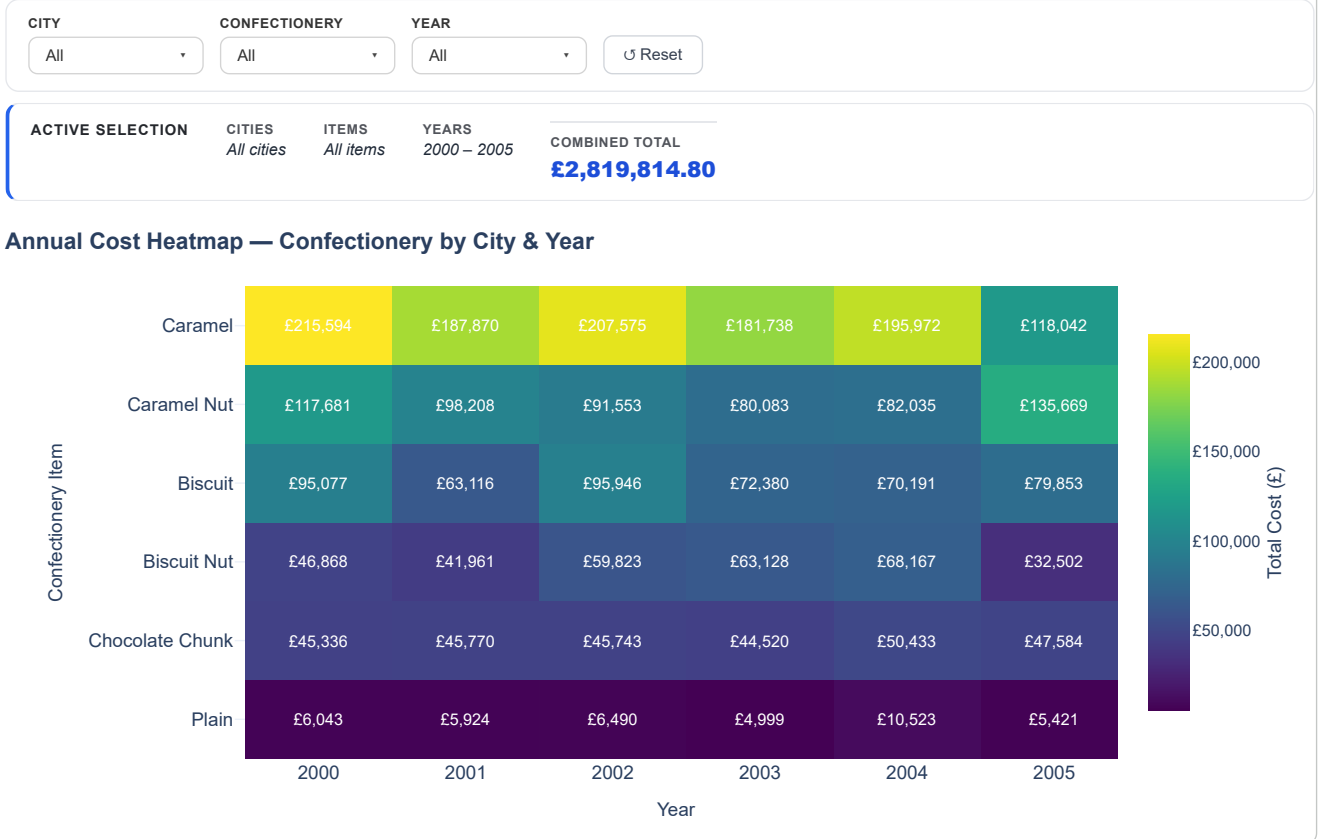
    renderAll();

    }}());
    ""

    ch_heatmap_html = pio.to_html(
        fig_heat,
        full_html=False,
        include_plotlyjs='cdn',
        config=dict(displaylogo=False, responsive=True),
        div_id='fig-heatmap',
        post_script=post_script,
    )

    display(HTML(ch_heatmap_html))

```



## 8. Chart 4 — Revenue by Confectionery per City (Spider - Radar)

```
# =====
# Chart 4 – Revenue by Confectionery per City (Spider - Radar)
# =====

sorted_cities_polar = sorted(bakery_dataset['City'].unique().tolist())
sorted_confs_polar = sorted(bakery_dataset['Confectionary'].unique().tolist())
all_years_polar = sorted(bakery_dataset['Year'].unique().tolist())

# — All-time totals (base figure) —————

conf_city_polar = bakery_dataset.groupby(
    ['City', 'Confectionary'], as_index=False
)['Revenue(£)'].sum()
conf_city_polar['City'] = pd.Categorical(
    conf_city_polar['City'], categories=sorted_cities_polar, ordered=True
)
conf_city_polar['Confectionary'] = pd.Categorical(
    conf_city_polar['Confectionary'], categories=sorted_confs_polar, ordered=True
)
conf_city_polar = conf_city_polar.sort_values(['City', 'Confectionary'])

# — Per-year lookup for JS year filter —————

polar_yr = {}
for _yr in all_years_polar:
    polar_yr[str(_yr)] = {}
    _yrd = bakery_dataset[bakery_dataset['Year'] == _yr]
    for _city in sorted_cities_polar:
        _cdf = _yrd[_yrd['City'] == _city]
        polar_yr[str(_yr)][_city] = [
            float(_cdf[_cdf['Confectionary'] == _conf]['Revenue(£)'].sum())
            for _conf in sorted_confs_polar
        ]

_polar_yr_js = json.dumps(polar_yr)
_years_polar_js = json.dumps([str(y) for y in all_years_polar])
_cities_polar_js = json.dumps(sorted_cities_polar)
```

```

fig4 = px.line_polar(
    conf_city_polar,
    r='Revenue(£)',
    theta='Confectionary',
    color='City',
    color_discrete_map=CITY_COLORS,
    line_close=True,
    template='plotly_white',
    title='<b>Revenue by Confectionery per City (Spider / Radar)</b>',
)
fig4.update_traces(
    fill='toself',
    opacity=0.62,
    hovertemplate=(
        '<b>{%fullData.name}</b><br>'
        '%{theta}<br>'
        'Revenue: £{%r:,.0f}'
        '<extra></extra>'
    ),
)
fig4.update_layout(
    title_font_size=18,
    font=dict(family='Arial', size=13, color='#333'),
    polar=dict(
        radialaxis=dict(
            visible=True,
            tickprefix='£',
            tickformat=',.0s',
            tickfont=dict(size=12, color='black', family='Arial'),
            gridcolor='#d4d4d4',
            linecolor='#d4d4d4',
        ),
        angularaxis=dict(
            tickfont=dict(size=14, color='black', family='Arial'),
            gridcolor='#d4d4d4',
            linecolor='#d4d4d4',
            direction='clockwise',
        ),
        bgcolor='#f9f9f9',
    ),
    legend=dict(title='City', x=1.08, y=0.5, yanchor='middle',
        font=dict(size=12, color='#333')),
    margin=dict(t=60, b=40, l=50, r=150),
    height=520,
)

_trace_order_polar = [t.name for t in fig4.data]
_trace_order_polar_js = json.dumps(_trace_order_polar)

ps4 = f"""
(function() {{
    var gd      = document.getElementById('fig4-polar');
    var yrData  = {_polar_yr_js};
    var allYears = {_years_polar_js};
    var traceOrder = {_trace_order_polar_js};
    var nConfs  = {len(sorted_confs_polar)};

    var selStyle = 'padding:6px 10px;border:1px solid #d0d0d0;border-radius:6px;font-size:13px;font-family:Arial,sans-serif;back
    var rstStyle = 'padding:6px 14px;background:#f4f4f4;color:#444;border:1px solid #d0d0d0;border-radius:6px;font-size:13px;fc

    var yrOpts = ['All'].concat(allYears).map(function(y){{
        return 'option value="'+y+'"' + (y==='All'? ' selected':'') + '>' + y + '</option>';
    }}).join('');

    var panel = document.createElement('div');
    panel.style.cssText = 'display:flex;align-items:center;gap:10px;padding:10px 16px;margin-bottom:6px;background:#fff;border:
    panel.innerHTML =
        '<span style="color:#555;font-weight:600;white-space:nowrap;">Filter</span>' +
        '<div style="width:1px;height:24px;background:#e2e2e2;margin:0 4px;"></div>' +
        '<span style="color:#777;font-size:12px;">Year</span>' +
        '<select id="f4-yr" style="'+selStyle+'">' + yrOpts + '</select>' +
        '<div style="width:1px;height:24px;background:#e2e2e2;margin:0 4px;"></div>' +
        '<button id="f4-rst" style="'+rstStyle+'">Reset</button>';
    gd.parentNode.insertBefore(panel, gd);

    function applyFilter() {{

```

```

var selYr = document.getElementById('f4-yr').value;
var newR = [];
traceOrder.forEach(function(city) {{
  if (selYr === 'All') {{
    var totals = allYears.reduce(function(acc, y) {{
      return acc.map(function(v, i) {{ return v + yrData[y][city][i]; }});
    }}, new Array(nConfs).fill(0));
    newR.push(totals);
  }} else {{
    newR.push(yrData[selYr][city]);
  }}
}});
Plotly.restyle(gd, {{ r: newR }});
}}

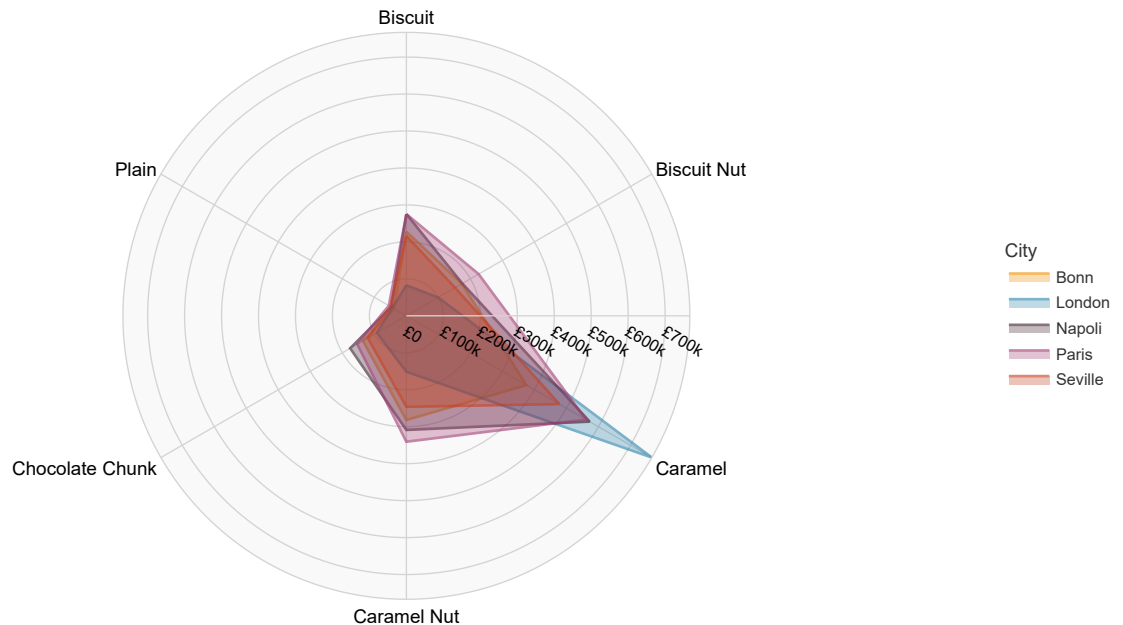
document.getElementById('f4-yr').addEventListener('change', applyFilter);
document.getElementById('f4-rst').addEventListener('click', function() {{
  document.getElementById('f4-yr').value = 'All'; applyFilter();
}});
}}());
"""

ch4_html = pio.to_html(
  fig4, full_html=False, include_plotlyjs='cdn',
  config=dict(displaylogo=False, responsive=True),
  div_id='fig4-polar', post_script=ps4,
)
display(HTML(ch4_html))

```

Filter Year All Reset

### Revenue by Confectionery per City (Spider / Radar)



## 9. Chart 5 — Annual Profit Margin (%) by City [Grouped Bar]

```

# =====
# Chart 5 - Annual Profit Margin (%) by City [Grouped Bar + Year Filter]
# =====

yearly_city = (
  bakery_dataset.groupby(['YearStr', 'City'], as_index=False)
  .agg(
    Revenue=('Revenue(£)', 'sum'),
    Profit=('Profit(£)', 'sum')
  )
)

```

```

    )
    yearly_city['Margin'] = (yearly_city['Profit'] / yearly_city['Revenue'] * 100).round(1)

num_cities = bakery_dataset['City'].nunique()
all_years = sorted(yearly_city['YearStr'].unique().tolist())

fig5 = px.bar(
    yearly_city,
    x='YearStr',
    y='Margin',
    color='City',
    color_discrete_map=CITY_COLORS,
    barmode='group',
    title='<b>Annual Profit Margin (%) by City</b>',
    labels={'YearStr': 'Year', 'Margin': 'Profit Margin (%)'},
    template='plotly_white',
)
fig5.update_traces(
    hovertemplate=(
        '<b>{fullData.name}</b><br>'
        'Year: {x}<br>'
        'Profit Margin: {y:.1f}%'
        '<extra></extra>'
    )
)
fig5.update_layout(
    title_font_size=18,
    font=dict(family='Arial', size=13),
    yaxis=dict(ticksuffix='%'),
    hovermode='closest',
    legend=dict(title='City'),
    margin=dict(t=30, b=80, l=70, r=30),
    height=480,
)

post_script = f"""
(function() {{
    var gd      = document.getElementById('fig5-bar');
    var allYears = {all_years};
    var numCities = {num_cities};

    var selectStyle = [
        'padding:6px 10px','border:1px solid #d0d0d0','border-radius:6px',
        'font-size:13px','font-family:Arial,sans-serif','background:#fafafa',
        'color:#333','cursor:pointer','outline:none',
    ].join(';');

    var resetBtnStyle = [
        'padding:6px 14px','background:#f4f4f4','color:#444',
        'border:1px solid #d0d0d0','border-radius:6px',
        'font-size:13px','font-family:Arial,sans-serif','cursor:pointer',
    ].join(';');

    var divider = '<div style="width:1px;height:24px;background:#e2e2e2;margin:0 4px;"></div>';

    var startOpts = allYears.map(function(y, i) {{
        return 'option value="'+i+'"' + (i===0? ' selected':'') + '>' + y + '</option>';
    }}).join('');

    var endOpts = allYears.map(function(y, i) {{
        return 'option value="'+i+'"' + (i===allYears.length-1? ' selected':'') + '>' + y + '</option>';
    }}).join('');

    var panel = document.createElement('div');
    panel.style.cssText = [
        'display:flex','align-items:center','gap:10px',
        'padding:10px 16px','margin-bottom:6px','background:#ffffff',
        'border:1px solid #e2e2e2','border-radius:10px',
        'font-family:Arial,sans-serif','font-size:13px',
        'box-sizing:border-box','width:100%',
    ].join(';');

    panel.innerHTML =
        '<span style="color:#555;font-weight:500;white-space:nowrap;">Year Range</span>' +
        divider +

```

```

'<label style="color:#777;white-space:nowrap;">From</label>' +
'<select id="f5-start" style="'+selectStyle+'">'+startOpts+'</select>' +
'<label style="color:#777;white-space:nowrap;">To</label>' +
'<select id="f5-end" style="'+selectStyle+'">'+endOpts+'</select>' +
divider +
'<button id="f5-reset" style="'+resetBtnStyle+'">Reset All</button>';

gd.parentNode.insertBefore(panel, gd);

var startEl = document.getElementById('f5-start');
var endEl = document.getElementById('f5-end');

function rebuildEndDropdown(selectedStartIdx) {{
    var prevEndIdx = parseInt(endEl.value);
    endEl.innerHTML = '';
    allYears.forEach(function(y, i) {{
        if (i >= selectedStartIdx) {{
            var opt = document.createElement('option');
            opt.value = i;
            opt.textContent = y;
            if (i === prevEndIdx && prevEndIdx >= selectedStartIdx) {{
                opt.selected = true;
            }} else if (prevEndIdx < selectedStartIdx && i === allYears.length - 1) {{
                opt.selected = true;
            }}
            endEl.appendChild(opt);
        }}
    }});
}}

function applyRange() {{
    var s = parseInt(startEl.value);
    var e = parseInt(endEl.value);
    Plotly.relayout(gd, {{'xaxis.range': [s - 0.5, e + 0.5]}});
}}

startEl.addEventListener('change', function() {{
    var s = parseInt(this.value);
    rebuildEndDropdown(s);
    applyRange();
}});

endEl.addEventListener('change', applyRange);

document.getElementById('f5-reset').addEventListener('click', function() {{
    startEl.value = '0';
    rebuildEndDropdown(0);
    endEl.value = String(allYears.length - 1);
    Plotly.relayout(gd, {{'xaxis.autorange': true}});
    var indices = Array.from({{length: numCities}}, function(_, i) {{ return i; }});
    Plotly.restyle(gd, {{'visible': true}}, indices);
}});
}}());
"""

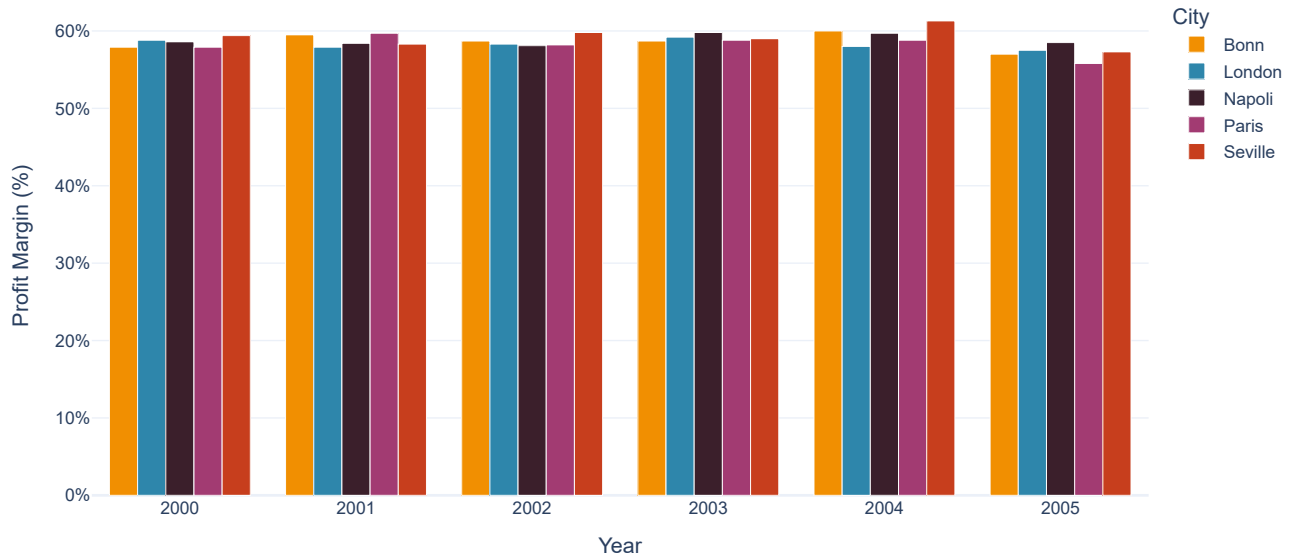
html_out = pio.to_html(
    fig5,
    full_html=False,
    include_plotlyjs='cdn',
    div_id='fig5-bar',
    post_script=post_script,
)

ps5 = post_script
display(HTML(html_out))

```

Year Range From  To

**Annual Profit Margin (%) by City**



## 10. Chart 6 — Units Sold vs Profit [Scatter + Filters]

```
# =====
# Chart 6 – Units Sold vs Profit
# =====

all_confs = ['All'] + sorted(bakery_dataset['Confectionary'].unique().tolist())
units_min = int(bakery_dataset['Units Sold'].min())
units_max = int(bakery_dataset['Units Sold'].max())

city_js_data = {}
for city in bakery_dataset['City'].unique():
    cdf = bakery_dataset[bakery_dataset['City'] == city].copy()
    city_js_data[city] = {
        'x': cdf['Units Sold'].tolist(),
        'y': cdf['Profit(£)'].tolist(),
        'conf': cdf['Confectionary'].tolist(),
        'date': cdf['Date'].dt.strftime('%Y-%m-%d').tolist(),
    }

fig6 = px.scatter(
    bakery_dataset,
    x='Units Sold',
    y='Profit(£)',
    color='City',
    symbol='City',
    color_discrete_map=CITY_COLORS,
    title='<b>Units Sold vs Profit – by City & Confectionery</b>',
    labels={'Units Sold': 'Units Sold', 'Profit(£)': 'Profit (£)'},
    template='plotly_white',
    opacity=0.78,
)

fig6.update_traces(
    marker=dict(size=8, line=dict(width=0.5, color='white')),
    hovertemplate=(
        '<b>{fullData.name}</b><br>'
        'Units Sold: {x:,><br>'
        'Profit: £{y:,.0f}<br>'
        'Confectionary: {customdata[0]}<br>'
        'Date: {customdata[1]}'
        '<extra></extra>'
    ),
),
    customdata=bakery_dataset[['Confectionary', 'Date']].assign(
        Date=bakery_dataset['Date'].dt.strftime('%Y-%m-%d')
    )
)
```

```

    date= bakery_dataset[ date ].strftime( '%Y-%m-%d ' )
    )[['Confectionary', 'Date']].values,
)

trace_city_order = [t.name for t in fig6.data]

fig6.update_layout(
    title_font_size=18,
    font=dict(family='Arial', size=13),
    yaxis=dict(tickprefix='£', tickformat=''),
    xaxis=dict(tickformat=''),
    legend=dict(
        title='City',
        orientation='v',
        x=1.02, y=0.5,
        yanchor='middle',
        bgcolor='rgba(255,255,255,0.9)',
        bordercolor='#dddddd',
        borderwidth=1,
    ),
    margin=dict(t=30, b=40, r=160, l=70),
    height=500,
)

post_script = f"""
(function() {{
    var gd      = document.getElementById('fig6-scatter');
    var cityData = {json.dumps(city_js_data)};
    var traceOrder = {json.dumps(trace_city_order)};
    var allConfs = {json.dumps(all_confs)};
    var unitsMin  = {units_min};
    var unitsMax  = {units_max};

    var selectStyle = [
        'padding:6px 10px', 'border:1px solid #d0d0d0', 'border-radius:6px',
        'font-size:13px', 'font-family:Arial,sans-serif', 'background:#fafafa',
        'color:#333', 'cursor:pointer', 'outline:none', 'min-width:150px',
    ].join(';');

    var resetBtnStyle = [
        'padding:6px 14px', 'background:#f4f4f4', 'color:#444',
        'border:1px solid #d0d0d0', 'border-radius:6px',
        'font-size:13px', 'font-family:Arial,sans-serif', 'cursor:pointer',
    ].join(';');

    var divider = '<div style="width:1px;height:24px;background:#e2e2e2;margin:0 4px;"></div>';

    var confOpts = allConfs.map(function(c) {{
        return '<option value="'+c+'">'+c+'</option>';
    }}).join('');

    var panel = document.createElement('div');
    panel.style.cssText = [
        'display:flex', 'flex-wrap:wrap', 'align-items:center', 'gap:10px',
        'padding:10px 16px', 'margin-bottom:6px', 'background:#ffffff',
        'border:1px solid #e2e2e2', 'border-radius:10px',
        'font-family:Arial,sans-serif', 'font-size:13px',
        'box-sizing:border-box', 'width:100%',
    ].join(';');

    panel.innerHTML =
        '<span style="color:#555;font-weight:500;white-space:nowrap;">Filters</span>' +
        divider +
        '<span style="color:#777;font-size:12px;">Confectionery</span>' +
        '<select id="f6-conf" style="'+selectStyle+'">'+confOpts+'</select>' +
        divider +
        '<div style="display:flex;flex-direction:column;gap:4px;min-width:200px;">' +
            '<div style="display:flex;justify-content:space-between;">' +
                '<span style="color:#777;font-size:12px;">Min Units Sold</span>' +
                '<span id="f6-min-label" style="font-size:12px;color:#3d7ebf;font-weight:500;">' +
                    unitsMin.toLocaleString() +
                '</span>' +
            '</div>' +
            '<input type="range" id="f6-slider"' +
                ' min="'+unitsMin+'" max="'+unitsMax+'" value="'+unitsMin+'" step="50"' +
                ' style="width:100%;accent-color:#3d7ebf;">' +
        '</div>' +

```



```

divider +
'<button id="f6-reset" style="'+resetBtnStyle+'">Reset All</button>';

gd.parentNode.insertBefore(panel, gd);

function applyFilters() {{
  var selConf = document.getElementById('f6-conf').value;
  var loUnits = parseInt(document.getElementById('f6-slider').value);
  var newX = [], newY = [], newCD = [];

  traceOrder.forEach(function(city) {{
    var d = cityData[city];
    var fx = [], fy = [], fcd = [];
    for (var i = 0; i < d.x.length; i++) {{
      var matchConf = (selConf === 'All') || (d.conf[i] === selConf);
      var matchUnits = d.x[i] >= loUnits;
      if (matchConf && matchUnits) {{
        fx.push(d.x[i]);
        fy.push(d.y[i]);
        fcd.push([d.conf[i], d.date[i]]);
      }}
    }}
    newX.push(fx);
    newY.push(fy);
    newCD.push(fcd);
  }});

  Plotly.restyle(gd, {{ x: newX, y: newY, customdata: newCD }});
}}

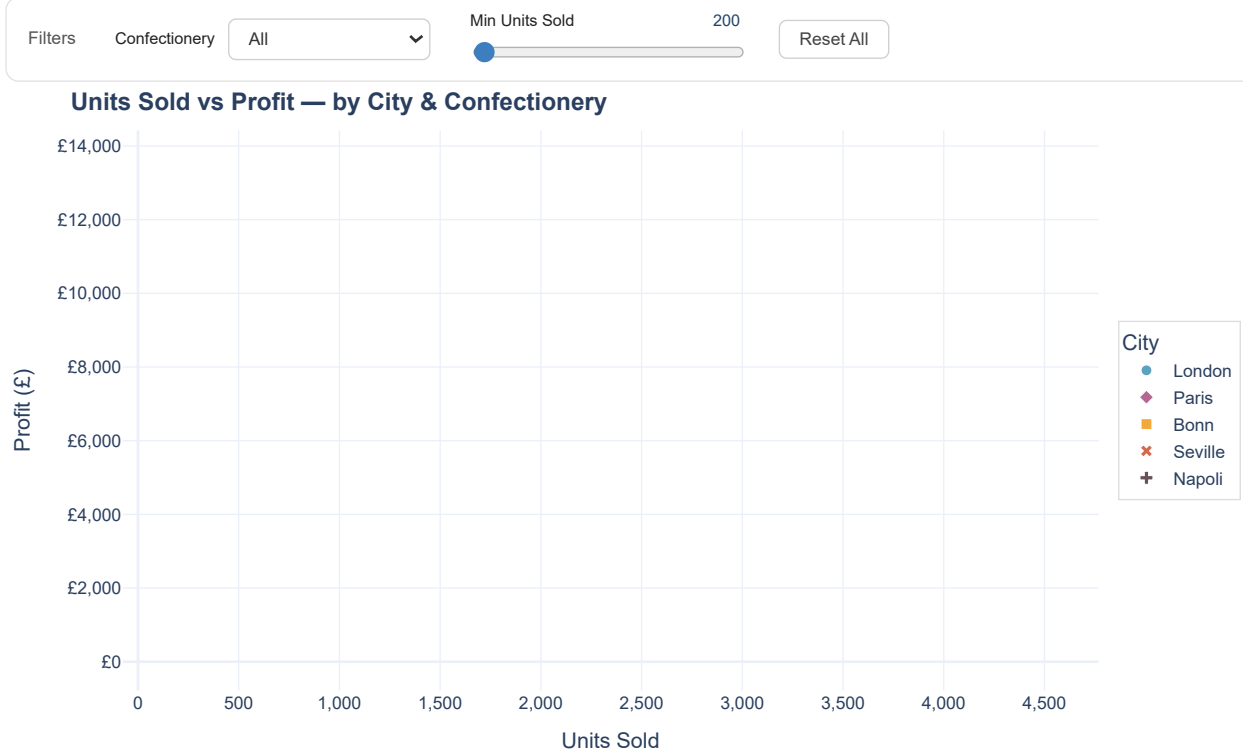
document.getElementById('f6-conf').addEventListener('change', applyFilters);

document.getElementById('f6-slider').addEventListener('input', function() {{
  document.getElementById('f6-min-label').textContent =
    parseInt(this.value).toLocaleString();
  applyFilters();
}});

document.getElementById('f6-reset').addEventListener('click', function() {{
  document.getElementById('f6-conf').value = 'All';
  document.getElementById('f6-slider').value = unitsMin;
  document.getElementById('f6-min-label').textContent = unitsMin.toLocaleString();
  var origX = [], origY = [], origCD = [];
  traceOrder.forEach(function(city) {{
    var d = cityData[city];
    origX.push(d.x);
    origY.push(d.y);
    origCD.push(d.x.map(function(_, i) {{ return [d.conf[i], d.date[i]]; }}));
  }});
  Plotly.restyle(gd, {{ x: origX, y: origY, customdata: origCD, visible: true }});
}});
}}());
"""

html_out = pio.to_html(
  fig6,
  full_html=False,
  include_plotlyjs='cdn',
  div_id='fig6-scatter',
  post_script=post_script,
)
ps6 = post_script
display(HTML(html_out))

```



## 11. Chart 7 — Revenue Treemap (City → Confectionery)

```
# =====
# Chart 7 – Revenue Breakdown: City → Confectionery (Treemap)
# =====

fig7 = px.treemap(
    bakery_dataset,
    path=[px.Constant('All Cities'), 'City', 'Confectionery'],
    values='Revenue(£)',
    color='City',
    color_discrete_map=CITY_COLORS,
    title='<b>Revenue Breakdown: City → Confectionery (Treemap)</b>',
    template='plotly_white',
)
fig7.update_traces(
    textinfo='label+percent parent',
    hovertemplate=(
        '<b>{label}</b><br>'
        'Revenue: £{value:,.0f}<br>'
        '{percentParent:.1%} of {parent}<br>'
        '{percentRoot:.1%} of all cities'
        '<extra></extra>'
    ),
    root_color='#f8fafc',
    marker=dict(line=dict(width=1.5, color='white')),
)
fig7.update_layout(
    title_font_size=18,
    font=dict(family='Arial', size=13),
    margin=dict(t=44, b=10, l=10, r=10),
    height=500,
)
ps7 = None

ch7_html = pio.to_html(
    fig7, full_html=False, include_plotlyjs='cdn',
    config=dict(displaylogo=False), div_id='fig7-treemap',
)
display(HTML(ch7_html))
```

## Revenue Breakdown: City → Confectionery (Treemap)



## 12. Chart 8 — Revenue Flow (City → Confectionery)

```
# =====
# Chart 8 – Revenue Sankey (city chips filter + product dropdown + click highlight)
# =====

def _hex_to_rgba(hex_col, alpha=0.50):
    h = hex_col.lstrip('#')
    r, g, b = int(h[0:2], 16), int(h[2:4], 16), int(h[4:6], 16)
    return f'rgba({r},{g},{b},{alpha})'

_cities_sk = sorted(bakery_dataset['City'].unique().tolist())
_confs_sk = sorted(bakery_dataset['Confectionary'].unique().tolist())
_n_cities = len(_cities_sk)

_cc_rev = bakery_dataset.groupby(
    ['City', 'Confectionary'], as_index=False
)['Revenue(£)'].sum()

_sk_links = []
for _, row in _cc_rev.iterrows():
    _sk_links.append({
        'city': row['City'],
        'conf': row['Confectionary'],
        'src': _cities_sk.index(row['City']),
        'tgt': _n_cities + _confs_sk.index(row['Confectionary']),
        'val': float(row['Revenue(£)']),
        'clr': _hex_to_rgba(CITY_COLORS.get(row['City'], '#888888'), 0.50),
    })

_node_labels = _cities_sk + _confs_sk
_node_colors = (
    [CITY_COLORS.get(c, '#888888') for c in _cities_sk] +
    ['#34d399' for _ in _confs_sk]
)

fig8 = go.Figure(data=[go.Sankey(
    arrangement='snap',
    node=dict(
        pad=18, thickness=22,
        line=dict(color='white', width=0.5),
        label=_node_labels,
        color=_node_colors,
        hovertemplate='<b>{label}</b><br>Revenue: £{value:,.0f}<extra></extra>',
    ),
```

```

link=dict(
    source=[l['src'] for l in _sk_links],
    target=[l['tgt'] for l in _sk_links],
    value=[l['val'] for l in _sk_links],
    color=[l['clr'] for l in _sk_links],
    hovertemplate=(
        '<b>{source.label}</b> → <b>{target.label}</b><br>'
        'Revenue: £{value:,.0f}'
        '<extra></extra>'
    ),
),
))
fig8.update_layout(
    title='<b>Revenue Flow: City → Confectionery (click a flow to lock-highlight it)</b>',
    title_font_size=18,
    font=dict(family='Arial', size=13, color='#444'),
    margin=dict(t=50, b=20, l=20, r=20),
    height=520,
    template='plotly_white',
    hoverlabel=dict(bgcolor='white', font_size=13, font_family='Arial'),
)

_sk_links_js = json.dumps(_sk_links)
_sk_cities_js = json.dumps(_cities_sk)
_sk_confs_all_js = json.dumps(['All'] + _confs_sk)
_node_labels_js = json.dumps(_node_labels)
_node_colors_js = json.dumps(_node_colors)
_city_colors_js = json.dumps(CITY_COLORS)

ps8 = f"""
(function() {{
    var gd = document.getElementById('fig8-sankey');
    var allLinks = {_sk_links_js};
    var allCities = {_sk_cities_js};
    var confOpts = {_sk_confs_all_js};
    var nodeLabels = {_node_labels_js};
    var nodeColors = {_node_colors_js};
    var cityColors = {_city_colors_js};

    var activeCities = new Set(allCities);
    var persistHlIdx = -1;

    // — Build filter panel —————

    var selStyle = 'padding:6px 10px;border:1px solid #d0d0d0;border-radius:6px;font-size:13px;font-family:Arial,sans-serif;
    var rstStyle = 'padding:6px 14px;background:#f4f4f4;color:#444;border:1px solid #d0d0d0;border-radius:6px;font-size:13px
    var chipBase = 'display:inline-block;padding:4px 12px;border-radius:20px;border:2px solid;font-size:11px;cursor:pointer;

    var confSel = '<select id="f8-conf" style="'+selStyle+'">'+
        confOpts.map(function(c){ return '<option value="'+c+'">'+c+'</option>'; }).join('')+
        '</select>';

    var chipsHtml = allCities.map(function(c) {{
        var clr = cityColors[c] || '#888';
        return '<span class="sk-chip" data-city="'+c+'" style="'+chipBase+'border-color:'+clr+';color:'+clr+'">'+c+'</span>';
    }}).join('');

    var panel = document.createElement('div');
    panel.style.cssText = 'display:flex;flex-wrap:wrap;align-items:center;gap:8px;padding:10px 16px;margin-bottom:6px;backgro
    panel.innerHTML =
        '<span style="color:#555;font-weight:600;white-space:nowrap;">Cities</span>' + chipsHtml +
        '<div style="width:1px;height:24px;background:#e2e2e2;margin:0 4px;"></div>' +
        '<span style="color:#777;font-size:12px;white-space:nowrap;">Product</span>' + confSel +
        '<div style="width:1px;height:24px;background:#e2e2e2;margin:0 4px;"></div>' +
        '<button id="f8-rst" style="'+rstStyle+'">Reset</button>' +
        '<span style="color:#aaa;font-size:11px;margin-left:4px;">#128161; Click a flow to lock-highlight</span>';
    gd.parentNode.insertBefore(panel, gd);

    // — Chip toggle —————

    panel.querySelectorAll('.sk-chip').forEach(function(chip) {{
        chip.style.opacity = '1';
        chip.addEventListener('click', function() {{
            var city = this.dataset.city;
            if (activeCities.has(city)) {{
                activeCities.delete(city);

```

```

        this.style.opacity = '0.25';
    } else {{
        activeCities.add(city);
        this.style.opacity = '1';
    }}
    persistHlIdx = -1;
    applyColors();
  }));
});

document.getElementById('f8-conf').addEventListener('change', function() {{
    persistHlIdx = -1; applyColors();
}});

document.getElementById('f8-rst').addEventListener('click', function() {{
    allCities.forEach(function(c) {{ activeCities.add(c); }});
    panel.querySelectorAll('.sk-chip').forEach(function(ch) {{ ch.style.opacity = '1'; }});
    document.getElementById('f8-conf').value = 'All';
    persistHlIdx = -1;
    applyColors();
}});

// — Color computation —————

function getLinkColors(hlIdx) {{
    var selConf = document.getElementById('f8-conf').value;
    return allLinks.map(function(l, i) {{
        var active = activeCities.has(l.city) && (selConf === 'All' || l.conf === selConf);
        if (!active) return 'rgba(210,210,210,0.05)';
        if (hlIdx === -1) return l.clr;
        // Corrected replacement for alpha value
        return i === hlIdx
            ? l.clr.replace(/\d+\.\d*$/, '0.92') // highlighted
            : l.clr.replace(/\d+\.\d*$/, '0.07'); // dimmed
    }});
}}

function getNodeColors() {{
    var selConf = document.getElementById('f8-conf').value;
    var active = new Set();
    allLinks.forEach(function(l) {{
        if (activeCities.has(l.city) && (selConf === 'All' || l.conf === selConf)) {{
            active.add(l.src); active.add(l.tgt);
        }}
    }});
    return nodeColors.map(function(c, i) {{
        return active.has(i) ? c : 'rgba(160,160,160,0.18)';
    }});
}}

function applyColors() {{
    Plotly.restyle(gd, {{
        'link.color': [getLinkColors(persistHlIdx)],
        'node.color': [getNodeColors()],
    }});
}}

// — Click = persistent highlight toggle —————

gd.on('plotly_click', function(data) {{
    if (!data.points || !data.points[0] || !data.points[0].source) return;
    var srcL = data.points[0].source.label, tgtL = data.points[0].target.label;
    var found = -1;
    allLinks.forEach(function(l, i) {{
        if (nodeLabels[l.src] === srcL && nodeLabels[l.tgt] === tgtL) found = i;
    }});
    if (found === -1) return;
    persistHlIdx = (persistHlIdx === found) ? -1 : found;
    applyColors();
}});

// — Hover = temporary highlight (only when no click-lock) —

gd.on('plotly_hover', function(data) {{
    if (persistHlIdx !== -1 || !data.points || !data.points[0] || !data.points[0].source) return;
    var srcL = data.points[0].source.label, tgtL = data.points[0].target.label;

```

```

var found = -1;
alllinks.forEach(function(l, i) {{
    if (nodeLabels[l.src] === srcL && nodeLabels[l.tgt] === tgtL) found = i;
}});
if (found !== -1) Plotly.restyle(gd, {{'link.color': [getLinkColors(found)]}});
}});

gd.on('plotly_unhover', function() {{
    if (persistHlIdx !== -1) return;
    applyColors();
}});
}}());
"""

ch8_html = pio.to_html(
    fig8, full_html=False, include_plotlyjs='cdn',
    config=dict(displaylogo=False, responsive=True),
    div_id='fig8-sankey', post_script=ps8,
)
display(HTML(ch8_html))

```

Cities Bonn London Napoli Paris Seville Product All Reset Click a flow to lock-highlight

### Revenue Flow: City → Confectionery (click a flow to lock-highlight it)



## 13. Dashboard

```

_cfg = dict(displaylogo=False, responsive=True)

# =====
# Dashboard
# =====
_sorted_cities = sorted(bakery_dataset["City"].unique().tolist())
_sorted_confs = sorted(bakery_dataset["Confectionary"].unique().tolist())
_sorted_years = sorted(bakery_dataset["Year"].unique().tolist())

_raw_df = bakery_dataset[
    ["Date", "City", "Confectionary", "Units Sold", "Revenue(£)", "Cost(£)", "Profit(£)", "Year"]
].copy()
_raw_df["Date"] = _raw_df["Date"].dt.strftime("%Y-%m-%d")
_raw_js = _raw_df.to_json(orient="records")

_ys_js = json.dumps(
    yearly_summary_df[
        ["Year", "City", "Revenue", "Profit", "Units", "Transactions", "lat", "lon", "Margin"]
    ].to_dict(orient="records")
)

```

```

)

_cc_yr = bakery_dataset.groupby(["Year", "City", "Confectionary"], as_index=False).agg(
    Revenue=("Revenue(£)", "sum"),
    Profit=("Profit(£)", "sum"),
    Units=("Units Sold", "sum"),
    Cost=("Cost(£)", "sum"),
)
_cc_js = json.dumps(_cc_yr.to_dict(orient="records"))

_heat_js = json.dumps(
    heat_agg[["Year", "City", "Confectionary", "Cost(£)"]].to_dict(orient="records")
)

_fig5_yr = bakery_dataset.groupby(["Year", "City"], as_index=False).agg(
    Revenue=("Revenue(£)", "sum"),
    Profit=("Profit(£)", "sum"),
)
_fig5_yr["Margin"] = (_fig5_yr["Profit"] / _fig5_yr["Revenue"] * 100).round(2)
_fig5_js = json.dumps(_fig5_yr[["Year", "City", "Margin"]].to_dict(orient="records"))

_scat = {}
for _c in _sorted_cities:
    _cdf = bakery_dataset[bakery_dataset["City"] == _c]
    _scat[_c] = {
        "x": _cdf["Units Sold"].tolist(),
        "y": _cdf["Profit(£)"].tolist(),
        "conf": _cdf["Confectionary"].tolist(),
        "year": _cdf["Year"].tolist(),
        "date": _cdf["Date"].dt.strftime("%Y-%m-%d").tolist(),
        "rev": _cdf["Revenue(£)"].tolist(),
        "cost": _cdf["Cost(£)"].tolist(),
    }
_scat_js = json.dumps(_scat)

_polar_js = json.dumps(polar_yr)
_confs_ord_js = json.dumps(sorted_confs_polar)

_cities_js = json.dumps(_sorted_cities)
_confs_js = json.dumps(_sorted_confs)
_years_js = json.dumps(_sorted_years)
_colors_js = json.dumps(CITY_COLORS)
_glob_max_js = json.dumps(float(global_max_revenue))

_tbl_json = _raw_df.to_json(orient="records")

# =====
# White theme figure copies
# =====
def _whiten(fig_in, h=420, mt=45, mb=40, ml=55, mr=25):
    f = copy.deepcopy(fig_in)
    f.update_layout(
        template="plotly_white",
        paper_bgcolor="#ffffff",
        plot_bgcolor="#ffffff",
        font=dict(color="#334155", family="Inter, Arial, sans-serif"),
        title_font=dict(color="#0f172a", size=15),
        height=h,
        margin=dict(t=mt, b=mb, l=ml, r=mr),
        legend=dict(
            font=dict(color="#475569"),
            bgcolor="rgba(255,255,255,0)",
            borderwidth=0,
        ),
    )
    f.update_xaxes(
        gridcolor="#e2e8f0",
        linecolor="#cbd5e1",
        zerolinecolor="#e2e8f0",
        tickfont=dict(color="#475569"),
        title_font=dict(color="#334155"),
    )
    f.update_yaxes(
        gridcolor="#e2e8f0",

```

```

        linecolor="#cbd5e1",
        zerolinecolor="#e2e8f0",
        tickfont=dict(color="#475569"),
        title_font=dict(color="#334155"),
    )
    try:
        f.update_annotations(font_color="#475569")
    except Exception:
        pass
    return f

_db1 = _whiten(fig1, h=590, mt=58, mb=25, ml=20, mr=70)
_db1.update_layout(width=None)
_db1.update_geos(
    bgcolor="#ffffff",
    landcolor="#f1f5f9",
    oceancolor="#e0f2fe",
    countrycolor="#94a3b8",
    coastlinecolor="#94a3b8",
    lakecolor="#e0f2fe",
)

for tr in _db1.data:
    if getattr(tr, "type", "") == "scattergeo" and "text" in str(getattr(tr, "mode", "")):
        tr.showlegend = False

_db2 = _whiten(fig2, h=455, mt=45, mb=10, ml=10, mr=10)
_db3 = _whiten(fig_heat, h=500, mt=50, mb=45, ml=165, mr=105)
_db4 = _whiten(fig4, h=520, mt=60, mb=80, ml=45, mr=110)

_db4.update_layout(
    title=dict(
        text="<b>Revenue by Confectionery per City (Spider / Radar)</b>",
        font=dict(size=18, color="#000000")
    ),
    polar=dict(
        bgcolor="#ffffff",
        radialaxis=dict(
            gridcolor="#d4b486",
            linecolor="#94a3b8",
            tickfont=dict(color="#000000", size=11),
            tickprefix="£",
            tickformat=",.0f",
            angle=90
        ),
        angularaxis=dict(
            gridcolor="#d4b486",
            linecolor="#94a3b8",
            tickfont=dict(color="#000000", size=14)
        ),
    ),
    legend=dict(
        title=dict(text="City", font=dict(color="#000000", size=13)),
        font=dict(color="#1e293b", size=13),
        x=1.08,
        y=0.5
    )
)

_db5 = _whiten(fig5, h=455, ml=65, mr=20, mb=55)
_db6 = _whiten(fig6, h=455, ml=65, mr=80, mb=45)
_db7 = _whiten(fig7, h=520, mt=45, mb=10, ml=10, mr=10)

_db8 = copy.deepcopy(fig8)
_db8.update_layout(
    template="plotly_white",
    paper_bgcolor="#ffffff",
    font=dict(color="#334155", family="Inter, Arial, sans-serif"),
    title="<b>Revenue Sankey: City → Confectionery</b>",
    title_font=dict(color="#0f172a", size=15),
    height=455,
    margin=dict(t=45, b=10, l=10, r=10),
)

# Fix hierarchy traces for white dashboard
for _f in [_db2, _db7]:
    for tr in f.data:

```



```

        if hasattr(_tr, "branchvalues"):
            _tr.branchvalues = "total"
        if hasattr(_tr, "textinfo"):
            _tr.textinfo = "label+percent parent"
        if hasattr(_tr, "insidetextfont"):
            _tr.insidetextfont = dict(color="#ffffff")

def _ch(fig, div_id):
    return pio.to_html(
        fig,
        full_html=False,
        include_plotlyjs=False,
        config=_cfg,
        div_id=div_id,
    )

_c1h = _ch(_db1, "db-c1")
_c2h = _ch(_db2, "db-c2")
_c3h = _ch(_db3, "db-c3")
_c4h = _ch(_db4, "db-c4")
_c5h = _ch(_db5, "db-c5")
_c6h = _ch(_db6, "db-c6")
_c7h = _ch(_db7, "db-c7")
_c8h = _ch(_db8, "db-c8")

# =====
# CSS
# =====
_CSS = """
:root{
  --bg:#f8fafc;
  --card:#ffffff;
  --soft:#f1f5f9;
  --line:#e2e8f0;
  --line2:#cbd5e1;
  --text:#0f172a;
  --muted:#64748b;
  --blue:#2563eb;
  --red:#e11d48;
}
*,*::before,*::after{box-sizing:border-box;margin:0;padding:0;}
body{
  font-family:'Inter',Arial,sans-serif;
  background:var(--bg);
  color:var(--text);
  min-height:100vh;
}
@keyframes fadeUp{from{opacity:0;transform:translateY(10px)}to{opacity:1;transform:translateY(0)}}

.hdr{
  background:linear-gradient(90deg,#ffffff,#eff6ff,#ffffff);
  border-bottom:1px solid var(--line);
  padding:18px 30px;
  display:flex;
  align-items:center;
  justify-content:space-between;
  animation:fadeUp .35s ease both;
}
.hl{display:flex;align-items:center;gap:14px;}
.hico{font-size:1.9rem;}
.htitle{font-size:1.28rem;font-weight:800;color:#0f172a;letter-spacing:-.35px;}
.hsub{font-size:.76rem;color:var(--muted);margin-top:3px;}
.hbadge{
  padding:5px 13px;
  border-radius:999px;
  background:#dbeafe;
  border:1px solid #bdfdbfe;
  color:#1d4ed8;
  font-size:11px;
  font-weight:700;
}

/* Important fix: high z-index and visible overflow */

```

```

.gtb{
  position:relative;
  z-index:99999;
  overflow:visible;
  display:flex;
  flex-wrap:wrap;
  align-items:flex-end;
  gap:12px;
  padding:14px 30px;
  background:#ffffff;
  border-bottom:1px solid var(--line);
  box-shadow:0 3px 14px rgba(15,23,42,.04);
}
.gfb-lbl{
  font-size:10px;
  font-weight:800;
  color:var(--muted);
  text-transform:uppercase;
  letter-spacing:.6px;
  display:block;
  margin-bottom:4px;
}
.dd-wrap{position:relative;display:inline-flex;flex-direction:column;overflow:visible;}
.dd-btn{
  padding:7px 11px 7px 12px;
  border:1px solid var(--line2);
  border-radius:9px;
  font-size:12px;
  background:#ffffff;
  color:#334155;
  cursor:pointer;
  outline:none;
  min-width:150px;
  display:flex;
  justify-content:space-between;
  align-items:center;
  gap:7px;
  white-space:nowrap;
  box-shadow:0 1px 2px rgba(15,23,42,.04);
}
.dd-btn:hover{border-color:var(--blue);color:#0f172a;}
.dd-menu{
  position:absolute;
  top:100%;
  left:0;
  z-index:100000;
  background:#ffffff;
  border:1px solid var(--line2);
  border-radius:10px;
  box-shadow:0 18px 40px rgba(15,23,42,.18);
  padding:7px 0;
  min-width:190px;
  max-height:280px;
  overflow-y:auto;
  display:none;
  margin-top:5px;
}
.dd-item{
  padding:7px 13px;
  font-size:12px;
  cursor:pointer;
  display:flex;
  align-items:center;
  gap:8px;
  color:#334155;
  white-space:nowrap;
}
.dd-item:hover{background:#eff6ff;color:#0f172a;}
.dd-item.head{border-bottom:1px solid var(--line);font-weight:800;color:#0f172a;}
.dd-item input[type=checkbox]{accent-color:var(--blue);cursor:pointer;}

.g-rst{
  padding:7px 16px;
  background:#fff1f2;
  color:var(--red);
  border:1px solid #fecdd3;
  border-radius:9px;
}

```

```

font-size:12px;
cursor:pointer;
font-weight:700;
align-self:flex-end;
}
.g-rst:hover{background:#ffe4e6;}

.kpi-grid{
position:relative;
z-index:1;
display:grid;
grid-template-columns:repeat(7,1fr);
gap:12px;
padding:14px 30px 16px;
background:#f8fafc;
border-bottom:1px solid var(--line);
}
.kpi{
background:#ffffff;
border:1px solid var(--line);
border-top:4px solid;
border-radius:14px;
padding:13px 14px;
box-shadow:0 5px 15px rgba(15,23,42,.05);
animation:fadeUp .45s ease both;
}
.kpi:hover{transform:translateY(-2px);transition:transform .15s;}
.kpi-ico{font-size:1.08rem;margin-bottom:5px;}
.kpi-lbl{font-size:9px;font-weight:800;color:var(--muted);text-transform:uppercase;letter-spacing:.7px;}
.kpi-val{font-size:1.12rem;font-weight:850;color:#0f172a;margin:5px 0 2px;white-space:nowrap;overflow:hidden;text-overflow:elli
.kpi-sub{font-size:10px;color:var(--muted);white-space:nowrap;overflow:hidden;text-overflow:ellipsis;}

.main{padding:16px 30px 30px;}
.grid{display:grid;gap:16px;}
.row-2{display:grid;grid-template-columns:1fr 1fr;gap:16px;}
.row-2-asm{display:grid;grid-template-columns:5fr 7fr;gap:16px;}
.card{
background:#ffffff;
border:1px solid var(--line);
border-radius:16px;
padding:8px 8px 6px;
display:flex;
flex-direction:column;
box-shadow:0 8px 24px rgba(15,23,42,.055);
transition:border-color .2s, box-shadow .2s;
animation:fadeUp .55s ease both .05s;
}
.card:hover{border-color:#bfdbfe;box-shadow:0 10px 28px rgba(37,99,235,.10);}

.tbl-wrap{overflow-x:auto;border-radius:12px;border:1px solid var(--line);}
.dtbl{width:100%;border-collapse:collapse;font-size:12px;}
.dtbl thead th{
background:#f1f5f9;
color:#475569;
padding:10px 14px;
text-align:left;
font-weight:800;
font-size:11px;
text-transform:uppercase;
letter-spacing:.45px;
border-bottom:1px solid var(--line);
cursor:pointer;
white-space:nowrap;
user-select:none;
}
.dtbl thead th:hover{color:#0f172a;}
.dtbl tbody tr{border-bottom:1px solid #eef2f7;transition:background .1s;}
.dtbl tbody tr:hover{background:#f8fafc;}
.dtbl tbody td{padding:9px 14px;color:#334155;white-space:nowrap;}
.tbl-ctrl{display:flex;align-items:center;gap:12px;padding:10px 0 12px;flex-wrap:wrap;}
.tbl-ctrl input{
padding:8px 12px;
border:1px solid var(--line2);
border-radius:8px;
background:#ffffff;
color:#334155;

```

```

    font-size:12px;
    outline:none;
    min-width:230px;
  }
  .tbl-btn{
    padding:7px 13px;
    border:1px solid var(--line2);
    border-radius:8px;
    background:#ffffff;
    color:#334155;
    font-size:12px;
    cursor:pointer;
  }
  .tbl-btn:hover{border-color:var(--blue);color:#1d4ed8;}
  .tbl-info{font-size:11px;color:var(--muted);}
  .sec-title{
    font-size:12px;
    font-weight:800;
    color:#475569;
    padding:12px 0 6px;
    letter-spacing:.35px;
    text-transform:uppercase;
  }
  .foot{
    text-align:center;
    padding:15px;
    font-size:10px;
    color:var(--muted);
    border-top:1px solid var(--line);
    background:#ffffff;
  }

  @media(max-width:1100px){
    .kpi-grid{grid-template-columns:repeat(3,1fr);}
    .row-2,.row-2-asym{grid-template-columns:1fr;}
  }
  @media(max-width:700px){
    .hdr{flex-direction:column;align-items:flex-start;gap:10px;}
    .kpi-grid{grid-template-columns:1fr;}
    .main,.gfb,.hdr{padding-left:16px;padding-right:16px;}
  }
  """

_KPI = """
<div class='kpi-grid'>
  <div class='kpi' style='border-top-color:#2563eb;'>
    <div class='kpi-ico'> 📈 </div><div class='kpi-lbl'>Total Revenue</div>
    <div class='kpi-val' id='kv-rev'>—</div><div class='kpi-sub' id='ks-tx'>— transactions</div>
  </div>
  <div class='kpi' style='border-top-color:#16a34a;'>
    <div class='kpi-ico'> 📊 </div><div class='kpi-lbl'>Total Profit</div>
    <div class='kpi-val' id='kv-prof'>—</div><div class='kpi-sub' id='ks-margin'>— avg margin</div>
  </div>
  <div class='kpi' style='border-top-color:#7c3aed;'>
    <div class='kpi-ico'> 📦 </div><div class='kpi-lbl'>Units Sold</div>
    <div class='kpi-val' id='kv-units'>—</div><div class='kpi-sub'>Across selection</div>
  </div>
  <div class='kpi' style='border-top-color:#f97316;'>
    <div class='kpi-ico'> 📈 </div><div class='kpi-lbl'>Profit Margin</div>
    <div class='kpi-val' id='kv-margin'>—</div><div class='kpi-sub'>Profit / revenue</div>
  </div>
  <div class='kpi' style='border-top-color:#e11d48;'>
    <div class='kpi-ico'> 📍 </div><div class='kpi-lbl'>Top City</div>
    <div class='kpi-val' id='kv-city'>—</div><div class='kpi-sub' id='ks-city'>By profit</div>
  </div>
  <div class='kpi' style='border-top-color:#0891b2;'>
    <div class='kpi-ico'> ⭐ </div><div class='kpi-lbl'>Top Product</div>
    <div class='kpi-val' id='kv-conf'>—</div><div class='kpi-sub' id='ks-conf'>By revenue</div>
  </div>
  <div class='kpi' style='border-top-color:#ca8a04;'>
    <div class='kpi-ico'> 🏙️ </div><div class='kpi-lbl'>Active Cities</div>
    <div class='kpi-val' id='kv-ncities'>—</div><div class='kpi-sub' id='ks-nyears'>— years</div>
  </div>
</div>
"""

```

```

_GLOBAL_JS = f"""
<script>
(function() {{

var rawData = {_raw_js};
var yearlyData = {_ys_js};
var ccData = {_cc_js};
var heatData = {_heat_js};
var fig5Data = {_fig5_js};
var scatData = {_scat_js};
var polarData = {_polar_js};
var confsOrder = {_confs_ord_js};
var allCities = {_cities_js};
var allConfs = {_confs_js};
var allYears = {_years_js};
var cityColors = {_colors_js};
var globalMaxRev = {_glob_max_js};

var selCities = allCities.slice();
var selConfs = allConfs.slice();
var selYears = allYears.slice();

function gbp(n) {{
  return '£' + Number(n || 0).toLocaleString('en-GB', {{maximumFractionDigits:0}});
}}
function gbpM(n) {{
  return '£' + (Number(n || 0) / 1e6).toFixed(2) + 'M';
}}
function yrLabel() {{
  if (selYears.length === allYears.length) return 'All Years';
  if (selYears.length === 1) return String(selYears[0]);
  return selYears.length + ' years';
}}

function selectedCCRows() {{
  return ccData.filter(function(d) {{
    return selCities.indexOf(d.City) !== -1 &&
      selConfs.indexOf(d.Confectionary) !== -1 &&
      selYears.indexOf(d.Year) !== -1;
  }});
}}

function aggCityConf() {{
  var m = {};
  selectedCCRows().forEach(function(d) {{
    var k = d.City + '|' + d.Confectionary;
    if (!m[k]) m[k] = {{City:d.City, Confectionary:d.Confectionary, Revenue:0, Profit:0, Units:0, Cost:0}};
    m[k].Revenue += Number(d.Revenue || 0);
    m[k].Profit += Number(d.Profit || 0);
    m[k].Units += Number(d.Units || 0);
    m[k].Cost += Number(d.Cost || 0);
  }});
  return Object.keys(m).map(function(k) {{ return m[k]; }});
}}

function makeDD(idBase, title, items, isNumeric, onchange) {{
  var wrap = document.createElement('div');
  wrap.className = 'dd-wrap';

  var lbl = document.createElement('span');
  lbl.className = 'gfb-lbl';
  lbl.textContent = title;
  wrap.appendChild(lbl);

  var btn = document.createElement('button');
  btn.className = 'dd-btn';
  btn.id = idBase + '-btn';
  btn.innerHTML = '<span id="' + idBase + '-lbl">All</span><span>*</span>';
  wrap.appendChild(btn);

  var menu = document.createElement('div');
  menu.className = 'dd-menu';
  menu.id = idBase + '-menu';

  var allRow = document.createElement('label');

```

```

allRow.className = 'dd-item head';
var allCb = document.createElement('input');
allCb.type = 'checkbox';
allCb.id = idBase + '-all';
allCb.checked = true;
allRow.appendChild(allCb);
allRow.appendChild(document.createTextNode('Select All'));
menu.appendChild(allRow);

items.forEach(function(v) {{
  var row = document.createElement('label');
  row.className = 'dd-item';
  var cb = document.createElement('input');
  cb.type = 'checkbox';
  cb.value = String(v);
  cb.className = idBase + '-cb';
  cb.checked = true;
  row.appendChild(cb);
  row.appendChild(document.createTextNode(String(v)));
  menu.appendChild(row);
}});

wrap.appendChild(menu);

btn.addEventListener('click', function(e) {{
  e.stopPropagation();
  document.querySelectorAll('.dd-menu').forEach(function(m) {{
    if (m !== menu) m.style.display = 'none';
  }});
  menu.style.display = menu.style.display === 'block' ? 'none' : 'block';
}});

document.addEventListener('click', function() {{
  menu.style.display = 'none';
}});

menu.addEventListener('click', function(e) {{
  e.stopPropagation();
}});

allCb.addEventListener('change', function() {{
  document.querySelectorAll('.' + idBase + '-cb').forEach(function(cb) {{
    cb.checked = allCb.checked;
  }});
  syncDD(idBase, items, isNumeric);
  onchange();
}});

menu.querySelectorAll('.' + idBase + '-cb').forEach(function(cb) {{
  cb.addEventListener('change', function() {{
    var checks = Array.from(document.querySelectorAll('.' + idBase + '-cb'));
    if (!checks.some(function(c) {{ return c.checked; }})) cb.checked = true;
    allCb.checked = checks.every(function(c) {{ return c.checked; }});
    syncDD(idBase, items, isNumeric);
    onchange();
  }});
}});

return wrap;
}}

function syncDD(idBase, items, isNumeric) {{
  var sel = Array.from(document.querySelectorAll('.' + idBase + '-cb'))
    .filter(function(cb) {{ return cb.checked; }})
    .map(function(cb) {{ return isNumeric ? Number(cb.value) : cb.value; }});

  if (idBase === 'dd-city') selCities = sel;
  else if (idBase === 'dd-conf') selConfs = sel;
  else selYears = sel;

  var lbl = document.getElementById(idBase + '-lbl');
  if (lbl) lbl.textContent = sel.length === items.length ? 'All' : sel.length + ' selected';
}}

function updateKPI() {{
  var rev = 0, profit = 0, units = 0, tx = 0;
  var cityProfit = {};

```

```

var selCities = [];
var confRev = {};

rawData.forEach(function(d) {{
  if (selCities.indexOf(d.City) !== -1 &&
      selConfs.indexOf(d.Confectionary) !== -1 &&
      selYears.indexOf(d.Year) !== -1) {{
    rev += Number(d['Revenue(£)'] || 0);
    profit += Number(d['Profit(£)'] || 0);
    units += Number(d['Units Sold'] || 0);
    tx += 1;
    cityProfit[d.City] = (cityProfit[d.City] || 0) + Number(d['Profit(£)'] || 0);
    confRev[d.Confectionary] = (confRev[d.Confectionary] || 0) + Number(d['Revenue(£)'] || 0);
  }}
}});

var margin = rev > 0 ? profit / rev * 100 : 0;
var topCity = Object.keys(cityProfit).sort(function(a,b) {{ return cityProfit[b] - cityProfit[a]; }})[0] || '-';
var topConf = Object.keys(confRev).sort(function(a,b) {{ return confRev[b] - confRev[a]; }})[0] || '-';

document.getElementById('kv-rev').textContent = gbpM(rev);
document.getElementById('kv-prof').textContent = gbpM(profit);
document.getElementById('kv-units').textContent = (units / 1000).toFixed(1) + 'K';
document.getElementById('kv-margin').textContent = margin.toFixed(1) + '%';
document.getElementById('kv-city').textContent = topCity;
document.getElementById('kv-conf').textContent = topConf;
document.getElementById('kv-ncities').textContent = selCities.length + ' / ' + allCities.length;
document.getElementById('ks-tx').textContent = tx.toLocaleString('en-GB') + ' transactions';
document.getElementById('ks-margin').textContent = margin.toFixed(1) + '% avg margin';
document.getElementById('ks-city').textContent = gbp(cityProfit[topCity] || 0) + ' profit';
document.getElementById('ks-conf').textContent = gbp(confRev[topConf] || 0) + ' revenue';
document.getElementById('ks-nyears').textContent = selYears.length + ' year' + (selYears.length > 1 ? 's' : '') + ' selected'
}}

function updateC1() {{
  var gd = document.getElementById('db-c1');
  if (!gd || !gd.data) return;

  var stats = {};
  allCities.forEach(function(city) {{
    stats[city] = {{rev:0, profit:0, units:0, margin:0, coord:null}};
  }});

  yearlyData.forEach(function(d) {{
    if (selYears.indexOf(d.Year) !== -1 && stats[d.City]) {{
      stats[d.City].rev += Number(d.Revenue || 0);
      stats[d.City].profit += Number(d.Profit || 0);
      stats[d.City].units += Number(d.Units || 0);
      if (!stats[d.City].coord) stats[d.City].coord = {{lat:d.lat, lon:d.lon}};
    }}
  }});

  allCities.forEach(function(city) {{
    var s = stats[city];
    s.margin = s.rev > 0 ? s.profit / s.rev * 100 : 0;
  }});

  var traces = JSON.parse(JSON.stringify(gd.data));
  allCities.forEach(function(city, i) {{
    var s = stats[city];
    var show = selCities.indexOf(city) !== -1 && s.rev > 0;
    var clr = cityColors[city] || '#64748b';
    var size = show ? (s.rev / globalMaxRev) * 50 + 14 : 12;

    var gi = i * 3, ri = i * 3 + 1, mi = i * 3 + 2;
    if (traces[gi]) {{
      traces[gi].visible = show;
      traces[gi].lat = [s.coord ? s.coord.lat : 0];
      traces[gi].lon = [s.coord ? s.coord.lon : 0];
      traces[gi].text = show ? ['<b>' + city + '</b><br>' + gbpM(s.rev)] : [''];
      traces[gi].textposition = city === 'Paris' ? 'bottom center' : 'top center';
      traces[gi].marker = {{size:size, color:clr, opacity:.88, line:{{width:2.2,color:'#ffffff'}}}};
      traces[gi].hovertemplate =
        '<b>' + city + '</b><br>Year: ' + yrLabel() +
        '<br>Revenue: ' + gbp(s.rev) +
        '<br>Profit: ' + gbp(s.profit) +
        '<br>Margin: ' + s.margin.toFixed(1) + '%' +

```

```

        '<br>Units: ' + Number(s.units).toLocaleString('en-GB') +
        '<extra></extra>';
    })
    if (traces[ri]) {{
        traces[ri].visible = show;
        traces[ri].x = [show ? s.rev : 0];
        traces[ri].text = [show ? gbpM(s.rev) : ''];
    }}
    if (traces[mi]) {{
        traces[mi].visible = show;
        traces[mi].x = [show ? s.margin : 0];
        traces[mi].text = [show ? s.margin.toFixed(1) + '%' : ''];
    }}
    });

    Plotly.react(gd, traces, gd.layout, {{displaylogo:false, responsive:true}});
    Plotly.relayout(gd, {{'title.text': '<b>European Bakery – City Performance Overview (' + yrLabel() + ')</b>'}});
}}

function updateC2() {{
    var gd = document.getElementById('db-c2');
    if (!gd || !gd.data) return;

    var rows = aggCityConf();
    var cityTotals = {};
    rows.forEach(function(d) {{
        cityTotals[d.City] = (cityTotals[d.City] || 0) + d.Revenue;
    }});

    var labels = ['All'];
    var parents = [''];
    var ids = ['root'];
    var values = [rows.reduce(function(a,d) {{ return a + d.Revenue; }}, 0)];
    var colors = ['#e2e8f0'];

    selCities.forEach(function(city) {{
        var total = cityTotals[city] || 0;
        if (total > 0) {{
            labels.push(city);
            parents.push('root');
            ids.push('city-' + city);
            values.push(total);
            colors.push(cityColors[city] || '#64748b');
        }}
    }});

    rows.forEach(function(d) {{
        if (d.Revenue > 0) {{
            labels.push(d.Confectionary);
            parents.push('city-' + d.City);
            ids.push('city-' + d.City + '-conf-' + d.Confectionary);
            values.push(d.Revenue);
            colors.push(cityColors[d.City] || '#64748b');
        }}
    }});

    var t = JSON.parse(JSON.stringify(gd.data[0]));
    t.type = 'sunburst';
    t.labels = labels;
    t.parents = parents;
    t.ids = ids;
    t.values = values;
    t.branchvalues = 'total';
    t.marker = {{colors:colors, line:{{color:'#ffffff', width:1}}}};
    t.textinfo = 'label+percent parent';
    t.hovertemplate = '<b>{{label}}</b><br>Revenue: £{{value:,.0f}}<extra></extra>';

    Plotly.react(gd, [t], gd.layout, {{displaylogo:false, responsive:true}});
}}

function updateC3() {{
    var gd = document.getElementById('db-c3');
    if (!gd || !gd.data) return;

    var xCities = selCities.slice();
    var yConfs = selConfs.slice();

```



```

var z = [];

yConfs.forEach(function(conf) {{
  var row = [];

  xCities.forEach(function(city) {{
    var v = 0;

    heatData.forEach(function(d) {{
      if (d.City === city &&
          d.Confectionary === conf &&
          selYears.indexOf(d.Year) !== -1) {{
        v += Number(d['Cost(£)'] || 0);
      }}
    }});

    row.push(v > 0 ? v : null);
  }});

  z.push(row);
}});

var annotations = [];

yConfs.forEach(function(conf, yi) {{
  xCities.forEach(function(city, xi) {{
    var v = z[yi][xi];

    if (v !== null) {{
      annotations.push({{
        x: city,
        y: conf,
        text: '£' + Number(v).toLocaleString('en-GB', {{maximumFractionDigits:0}}),
        showarrow: false,
        font: {{
          color: 'ffffff',
          size: 12
        }}
      }});
    }}
  }});
}});

var t = JSON.parse(JSON.stringify(gd.data[0]));
t.type = 'heatmap';
t.x = xCities;
t.y = yConfs;
t.z = z;
t.colorscales = 'Viridis';
t.hovertemplate =
  '<b>{{y}}</b><br>' +
  'City: {{x}}<br>' +
  'Cost: £{{z:.0f}}<extra></extra>';

var layout = JSON.parse(JSON.stringify(gd.layout));
layout.annotations = annotations;
layout.height = 520;
layout.margin = {{t:55, b:65, l:160, r:100}};
layout.title = {{
  text: '<b>Total Cost Heatmap by City and Confectionery</b>',
  font: {{color:'#000000', size:16}}
}};

layout.xaxis = {{
  title: {{text:'City', font:{{color:'#000000'}}}},
  type: 'category',
  tickfont: {{color:'#000000', size:12}},
  side: 'bottom'
}};

layout.yaxis = {{
  title: {{text:'Confectionery Item', font:{{color:'#000000'}}}},
  type: 'category',
  tickfont: {{color:'#000000', size:12}},
  automargin: true
}};

```

```

Plotly.react(gd, [t], layout, {{displaylogo:false, responsive:true}});
setTimeout(function() {{

var anns = gd.querySelectorAll('.annotation-text');

anns.forEach(function(a) {{
  a.style.fill = '#ffffff';

  a.style.textShadow = `
    -0.5px -0.5px 0 #000,
    0.5px -0.5px 0 #000,
    -0.5px 0.5px 0 #000,
    0.5px 0.5px 0 #000
  `;

  a.style.fontWeight = '700';
}}});

}}, 120);
}}

function updateC4() {{
  var gd = document.getElementById('db-c4');
  if (!gd || !gd.data) return;

  var traces = JSON.parse(JSON.stringify(gd.data));

  allCities.forEach(function(city, i) {{
    if (!traces[i]) return;

    var show = selCities.indexOf(city) !== -1;
    traces[i].visible = show ? true : 'legendonly';

    var rVals = new Array(confsOrder.length).fill(0);

    selYears.forEach(function(yr) {{
      var yd = polarData[String(yr)];
      if (yd && yd[city]) {{
        yd[city].forEach(function(v,j) {{
          rVals[j] += Number(v || 0);
        }});
      }}
    }});

    var theta = [];
    var r = [];

    confsOrder.forEach(function(conf, j) {{
      if (selConfs.indexOf(conf) !== -1) {{
        theta.push(conf);
        r.push(rVals[j]);
      }}
    }});

    if (theta.length > 0) {{
      theta.push(theta[0]);
      r.push(r[0]);
    }}

    traces[i].theta = theta;
    traces[i].r = r;

    traces[i].hovertemplate =
      '<b>' + city + '</b><br>' +
      'Confectionery: %{{theta}}<br>' +
      'Revenue: £%{{r:.0f}}<extra></extra>';
  }});

  var layout = JSON.parse(JSON.stringify(gd.layout));

  layout.height = 540;
  layout.margin = {{t:70, b:95, l:55, r:130}};

  if (!layout.polar) layout.polar = {{}};
  if (!layout.polar.radialaxis) layout.polar.radialaxis = {{}};
  if (!layout.polar.angularaxis) layout.polar.angularaxis = {{}};

```

```

1: (layout.polar.angularaxis, layout.polar.angularaxis = {}),

layout.polar.radialaxis.tickprefix = 'f';
layout.polar.radialaxis.tickformat = ',.0f';
layout.polar.radialaxis.tickfont = {{color:'#000000', size:11}};
layout.polar.radialaxis.gridcolor = '#d8eaf8';
layout.polar.radialaxis.linecolor = '#94a3b8';

layout.polar.angularaxis.tickfont = {{color:'#000000', size:14}};
layout.polar.angularaxis.gridcolor = '#d8eaf8';
layout.polar.angularaxis.linecolor = '#94a3b8';

layout.legend = {{
  title: {{text:'City', font:{{color:'#000000', size:13}}}},
  font: {{color:'#1e293b', size:13}},
  x: 1.08,
  y: 0.5
}};

Plotly.react(gd, traces, layout, {{displaylogo:false, responsive:true}});
}}

function updateC5() {{
  var gd = document.getElementById('db-c5');
  if (!gd || !gd.data) return;

  var sortedYrs = selYears.slice().sort(function(a,b) {{ return a-b; }});
  var traces = JSON.parse(JSON.stringify(gd.data));

  allCities.forEach(function(city, i) {{
    if (!traces[i]) return;
    var show = selCities.indexOf(city) !== -1;
    traces[i].visible = show ? true : 'legendonly';

    var yv = [];
    sortedYrs.forEach(function(yr) {{
      var row = fig5Data.find(function(d) {{ return d.City === city && d.Year === yr; }});
      yv.push(row ? row.Margin : null);
    }});

    traces[i].x = sortedYrs.map(String);
    traces[i].y = yv;
    traces[i].text = yv.map(function(v) {{ return v === null ? '' : v.toFixed(1) + '%'; }});
    traces[i].hovertemplate = '<b>' + city + '</b><br>Year: %{{x}}<br>Margin: %{{y:.1f}}%<extra></extra>';
  }});

  Plotly.react(gd, traces, gd.layout, {{displaylogo:false, responsive:true}});
}}

function updateC6() {{
  var gd = document.getElementById('db-c6');
  if (!gd || !gd.data) return;

  var traces = JSON.parse(JSON.stringify(gd.data));

  allCities.forEach(function(city, i) {{
    if (!traces[i]) return;
    var show = selCities.indexOf(city) !== -1;
    traces[i].visible = show ? true : 'legendonly';

    var cd = scatData[city];
    var x = [], y = [], custom = [];
    if (cd) {{
      for (var k = 0; k < cd.x.length; k++) {{
        if (selConfs.indexOf(cd.conf[k]) !== -1 &&
            selYears.indexOf(cd.year[k]) !== -1) {{
          x.push(cd.x[k]);
          y.push(cd.y[k]);
          custom.push([cd.conf[k], cd.date[k], cd.rev[k], cd.cost[k]]);
        }}
      }}
    }}
  }}

  traces[i].x = x;
  traces[i].y = y;
  traces[i].customdata = custom;
  traces[i].hovertemplate =

```

```

'<b>' + city + '</b><br>' +
'Product: %{{customdata[0]}}<br>' +
'Date: %{{customdata[1]}}<br>' +
'Units: %{{x:,}}<br>' +
'Profit: £%{{y:,.0f}}<br>' +
'Revenue: £%{{customdata[2]:,.0f}}<br>' +
'Cost: £%{{customdata[3]:,.0f}}<extra></extra>';
}});

Plotly.react(gd, traces, gd.layout, {{displaylogo:false, responsive:true}});
}}

function updateC7() {{
  var gd = document.getElementById('db-c7');
  if (!gd || !gd.data) return;

  var rows = aggCityConf();
  var cityTotals = {};
  rows.forEach(function(d) {{
    cityTotals[d.City] = (cityTotals[d.City] || 0) + d.Revenue;
  }});

  var labels = ['All Cities'];
  var parents = [''];
  var ids = ['root'];
  var values = [rows.reduce(function(a,d) {{ return a + d.Revenue; }}, 0)];
  var colors = ['#e2e8f0'];

  selCities.forEach(function(city) {{
    var total = cityTotals[city] || 0;
    if (total > 0) {{
      labels.push(city);
      parents.push('root');
      ids.push('city-' + city);
      values.push(total);
      colors.push(cityColors[city] || '#64748b');
    }}
  }});

  rows.forEach(function(d) {{
    if (d.Revenue > 0) {{
      labels.push(d.Confectionary);
      parents.push('city-' + d.City);
      ids.push('city-' + d.City + '-conf-' + d.Confectionary);
      values.push(d.Revenue);
      colors.push(cityColors[d.City] || '#64748b');
    }}
  }});

  var t = JSON.parse(JSON.stringify(gd.data[0]));
  t.type = 'treemap';
  t.labels = labels;
  t.parents = parents;
  t.ids = ids;
  t.values = values;
  t.branchvalues = 'total';
  t.marker = {{colors:colors, line:{{color:'#ffffff', width:1}}}};
  t.textinfo = 'label+value+percent parent';
  t.hovertemplate = '<b>%{{label}}</b><br>Revenue: £%{{value:,.0f}}<extra></extra>';

  Plotly.react(gd, [t], gd.layout, {{displaylogo:false, responsive:true}});
}}

function updateC8() {{
  var gd = document.getElementById('db-c8');
  if (!gd || !gd.data) return;

  var rows = aggCityConf();
  var activeCities = selCities.filter(function(city) {{
    return rows.some(function(d) {{ return d.City === city && d.Revenue > 0; }});
  }});
  var activeConfs = selConfs.filter(function(conf) {{
    return rows.some(function(d) {{ return d.Confectionary === conf && d.Revenue > 0; }});
  }});

  var nodeLabels = activeCities.concat(activeConfs);
  var nodeColors = activeCities.map(function(c) {{ return cityColors[c] || '#64748b'; }})

```

```

var nodeColors = activeCities.map(function(c) {{ return cityColors[c] || '#04748b' ; }})
    .concat(activeConfs.map(function() {{ return '#38bdf8'; }}}));

var src = [], tgt = [], val = [], col = [], custom = [];
rows.forEach(function(d) {{
  if (d.Revenue > 0) {{
    var s = activeCities.indexOf(d.City);
    var t = activeCities.length + activeConfs.indexOf(d.Confectionary);
    if (s >= 0 && t >= activeCities.length) {{
      src.push(s);
      tgt.push(t);
      val.push(d.Revenue);
      custom.push([d.Profit, d.Units, d.Cost]);
      var h = (cityColors[d.City] || '#64748b').replace('#','');
      var r = parseInt(h.substring(0,2),16);
      var g = parseInt(h.substring(2,4),16);
      var b = parseInt(h.substring(4,6),16);
      col.push('rgba(' + r + ',' + g + ',' + b + ',0.45)');
    }}
  }}
  }});

var newTrace = {{
  type:'sankey',
  arrangement:'snap',
  node:{{
    pad:18,
    thickness:22,
    line:{{color:'#ffffff', width:.7}},
    label:nodeLabels,
    color:nodeColors,
    hovertemplate:'<b>{{label}}</b><br>Total Revenue: £{{value:,.0f}}<extra></extra>'
  }},
  link:{{
    source:src,
    target:tgt,
    value:val,
    color:col,
    customdata:custom,
    hovertemplate:
      '<b>{{source.label}} → {{target.label}}</b><br>' +
      'Revenue: £{{value:,.0f}}<br>' +
      'Profit: £{{customdata[0]:,.0f}}<br>' +
      'Units: {{customdata[1]:,.0f}}<br>' +
      'Cost: £{{customdata[2]:,.0f}}<extra></extra>'
  }}
  }});

Plotly.react(gd, [newTrace], gd.layout, {{displaylogo:false, responsive:true}});
gd._sankeyOriginalColors = col.slice();
gd._sankeySelected = null;

if (!gd._sankeyEventsAttached) {{

  gd.on('plotly_hover', function(ev) {{
    if (gd._sankeySelected !== null) return;
    if (!ev.points || ev.points.length === 0) return;

    var p = ev.points[0];

    if (p.pointNumber === undefined) return;

    var newColors = gd._sankeyOriginalColors.map(function(c, i) {{
      return i === p.pointNumber
        ? c.replace('0.45', '0.95')
        : c.replace('0.45', '0.08');
    }});

    Plotly.restyle(gd, {{'link.color':[newColors]}}, [0]);
  }});

  gd.on('plotly_unhover', function() {{
    if (gd._sankeySelected !== null) return;
    Plotly.restyle(gd, {{'link.color':[gd._sankeyOriginalColors]}}, [0]);
  }});

  gd.on('plotly_click', function(ev) {{

```

```

    if (!ev.points || ev.points.length === 0) return;

    var p = ev.points[0];

    if (p.pointNumber === undefined) return;

    gd._sankeySelected = p.pointNumber;

    var newColors = gd._sankeyOriginalColors.map(function(c, i) {{
        return i === p.pointNumber
            ? c.replace('0.45', '1.0')
            : c.replace('0.45', '0.06');
    }});

    Plotly.restyle(gd, {'link.color':[newColors]}}, [0]);

    var info =
        '<b>' + p.source.label + ' → ' + p.target.label + '</b><br>' +
        'Revenue: £' + Number(p.value).toLocaleString('en-GB');

    var box = document.getElementById('sankey-info-box');
    if (box) {{
        box.innerHTML = info;
        box.style.display = 'block';
    }}
    });

    gd.addEventListener('dblclick', function() {{
        gd._sankeySelected = null;
        Plotly.restyle(gd, {'link.color':[gd._sankeyOriginalColors]}}, [0]);

        var box = document.getElementById('sankey-info-box');
        if (box) {{
            box.style.display = 'none';
            box.innerHTML = '';
        }}
    });

    gd._sankeyEventsAttached = true;
    }}
    }}

function applyAll() {{
    updateKPI();
    updateC1();
    updateC2();
    updateC3();
    updateC4();
    updateC5();
    updateC6();
    updateC7();
    updateC8();
}}

var gfb = document.getElementById('gfb');
if (gfb) {{
    gfb.appendChild(makeDD('dd-city', 'City', allCities, false, applyAll));
    gfb.appendChild(makeDD('dd-conf', 'Confectionery', allConfs, false, applyAll));
    gfb.appendChild(makeDD('dd-year', 'Year', allYears, true, applyAll));

    var rst = document.createElement('button');
    rst.className = 'g-rst';
    rst.textContent = '⌛ Reset All';
    rst.addEventListener('click', function() {{
        ['dd-city', 'dd-conf', 'dd-year'].forEach(function(id) {{
            document.querySelectorAll('.' + id + '-cb, #' + id + '-all').forEach(function(cb) {{
                cb.checked = true;
            }});
        }});
        var lbl = document.getElementById(id + '-lbl');
        if (lbl) lbl.textContent = 'All';
    }});
    selCities = allCities.slice();
    selConfs = allConfs.slice();
    selYears = allYears.slice();
    applyAll();
    });
    -fb -----

```

```

        gro.appendChild(rst);
    })

    setTimeout(applyAll, 700);
    window.addEventListener('resize', function() {{
        ['db-c1', 'db-c2', 'db-c3', 'db-c4', 'db-c5', 'db-c6', 'db-c7', 'db-c8'].forEach(function(id) {{
            var gd = document.getElementById(id);
            if (gd) Plotly.Plots.resize(gd);
        }});
    }});

    }}());
</script>
"""

_TABLE_JS = (
    "<script>\n"
    "var _TD = " + _tbl_json + ";\n"
    "var _tPage=1,_tPP=20,_tSort=null,_tDir=1,_tQ='';\n"
    "var _tCols=['Date','City','Confectionary','Units Sold','Revenue(£)','Cost(£)','Profit(£)','Year'];\n"
    "var _tFmt={'Revenue(£)':true,'Cost(£)':true,'Profit(£)':true};\n"
    "function _tRender(){\n"
    "    var data=_TD.filter(function(r){\n"
    "        if(!_tQ)return true;\n"
    "        return _tCols.some(function(c){return String(r[c]).toLowerCase().includes(_tQ)});\n"
    "    });\n"
    "    if(_tSort!=null){\n"
    "        var k=_tCols[_tSort];\n"
    "        data=data.slice().sort(function(a,b){\n"
    "            var av=a[k],bv=b[k];\n"
    "            if(!isNaN(av)&&!isNaN(bv))return(parseFloat(av)-parseFloat(bv))*_tDir;\n"
    "            return String(av).localeCompare(String(bv))*_tDir;\n"
    "        });\n"
    "    }\n"
    "    var tp=Math.max(1,Math.ceil(data.length/_tPP));\n"
    "    if(_tPage>tp)_tPage=tp;\n"
    "    var st=(_tPage-1)*_tPP,sl=data.slice(st,st+_tPP);\n"
    "    var hdr=_tCols.map(function(c,i){\n"
    "        var arr=_tSort===i?(_tDir>0?'▲':'▼'):'$';\n"
    "        return '<th onclick=\"_tSortCol('+i+')\">'+c+arr+'</th>';\n"
    "    }).join('');\n"
    "    var rows=sl.map(function(r){\n"
    "        return '<tr>'+_tCols.map(function(c){\n"
    "            var v=r[c];\n"
    "            if(_tFmt[c])return '<td>£'+parseFloat(v).toLocaleString('en-GB',{minimumFractionDigits:2,maximumFractionDigits:2})+'\n"
    "            return '<td>'+v+'</td>';\n"
    "        }).join('')+'</tr>';\n"
    "    }).join('');\n"
    "    document.getElementById('_tbl').innerHTML='<div class=\"tbl-wrap\"><table class=\"dtbl\"><thead><tr>'+hdr+'</tr></thead>\n"
    "    document.getElementById('_tpag').innerHTML='<button class=\"tbl-btn\" onclick=\"_tChangePage(-1)\">&#8592; Prev</button>\n"
    "    '<span class=\"tbl-info\">Page '+_tPage+' of '+tp+' &nbsp;&nbsp;'+data.length+' rows</span>'+\n"
    "    '<button class=\"tbl-btn\" onclick=\"_tChangePage(1)\">Next &#8594;</button>';\n"
    "    }\n"
    "    function _tSortCol(i){if(_tSort===i)_tDir*=-1;else{_tSort=i;_tDir=1;}_tPage=1;_tRender();}\n"
    "    function _tChangePage(d){_tPage+=d;_tRender();}\n"
    "    document.addEventListener('DOMContentLoaded',function(){\n"
    "        _tRender();\n"
    "    });\n"
    "    "</script>\n"
)

_dashboard_html = (
    "<!DOCTYPE html><html lang='en'><head>\n"
    "<meta charset='utf-8'><meta name='viewport' content='width=device-width,initial-scale=1'>\n"
    "<title>European Bakery Sales Dashboard – COM7021</title>\n"
    "<script src='https://cdn.plot.ly/plotly-latest.min.js'></script>\n"
    "<link href='https://fonts.googleapis.com/css2?family=Inter:wght@300;400;500;600;700;800&display=swap' rel='stylesheet'>\n"
    "<style>"+_CSS+"</style>\n"
    "</head><body>\n"
    "<div class='hdr'><div class='hl'><span class='hico'>🍞</span><div>\n"
    "<div class='htitle'>European Bakery Sales Dashboard</div>\n"
    "<div class='hsub'>COM7021 &nbsp;&nbsp;&nbsp; Muhammad Sameer (24169956) &nbsp;&nbsp;&nbsp; Arden University</div>\n"
    "</div></div><div class='hbadge'>2000 – 2005</div></div>\n"
    "<div class='gfb' id='gfb'></div>\n"
    "+_KPI +

```

```

"<div class='main'><div class='grid'>"
"<div class='card' style='margin-bottom:16px;'>" + _c1h + "</div>"
"<div class='row-2-asym' style='margin-bottom:16px;'>"
"<div class='card'>" + _c2h + "</div>"
"<div class='card'>" + _c4h + "</div>"
"</div>"
"<div class='card' style='margin-bottom:16px;'>" + _c7h + "</div>"
"<div class='row-2' style='margin-bottom:16px;'>"
"<div class='card'>" + _c6h + "</div>"
"<div class='card'>" + _c5h + "</div>"
"</div>"
"<div class='card' style='margin-bottom:16px;'>" + _c3h + "</div>"
"<div class='card' style='margin-bottom:16px;'>"
"<div id='sankey-info-box' style='display:none;margin:8px 10px 0;padding:10px 12px;border:1px solid #cbd5e1;border-radius:1"
+ _c8h +
"</div>"
"<div class='card' style='padding:16px 20px;margin-bottom:0;'>"
"<div class='sec-title'> 📄 Full Dataset – Sortable & Searchable</div>"
"<div class='tbl-ctrl'>"
"<span id='_tpag'></span>"
"</div>"
"<div id='_tbl'></div>"
"</div>"
"</div></div>"
"<div class='foot'>European Bakery Sales (2000–2005) & COM7021 Data Visualisation & Muhammad Sameer 24169956"
+ _TABLE_JS
+ _GLOBAL_JS
+ "</body></html>"
)

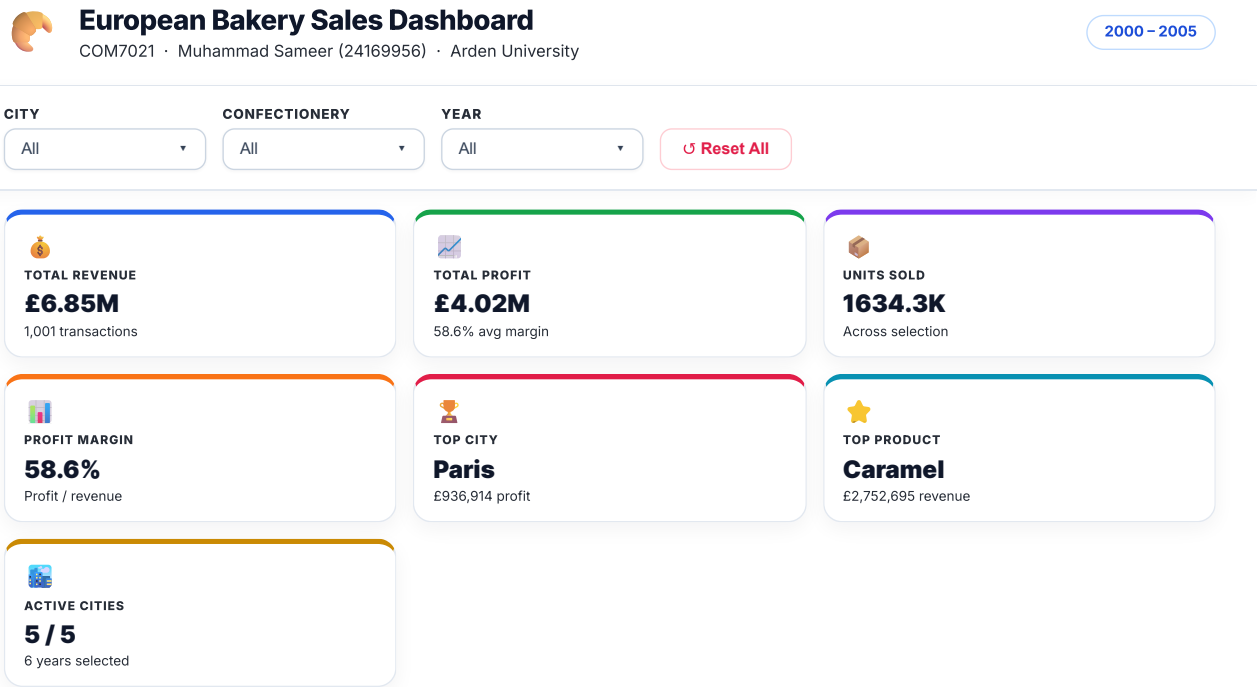
display(HTML(_dashboard_html))

with open("dashboard.html", "w", encoding="utf-8") as _f:
    _f.write(_dashboard_html)

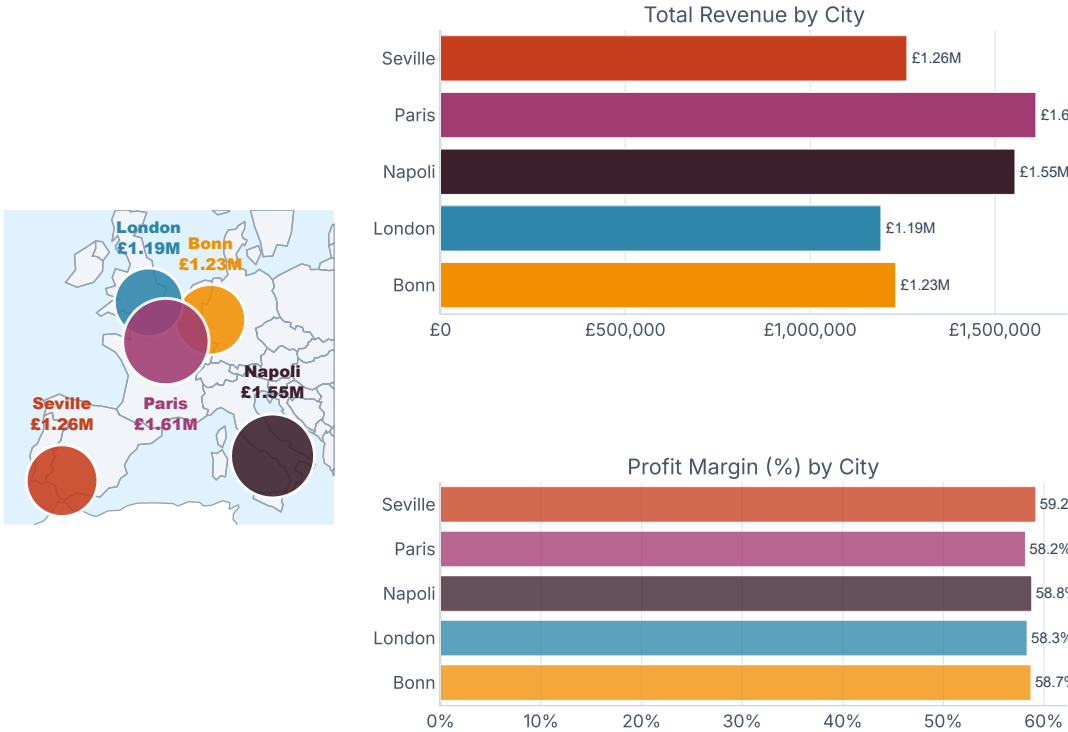
```



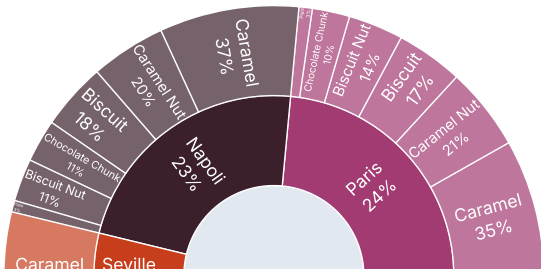


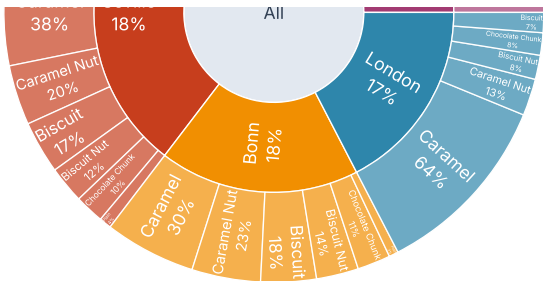


European Bakery — City Performance Overview (All Years)

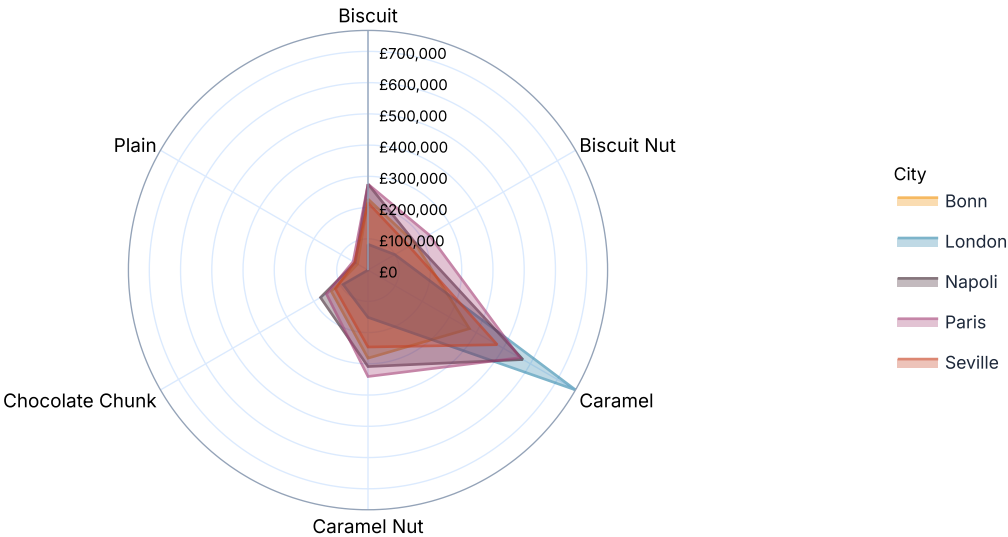


Revenue Sunburst: City → Confectionery

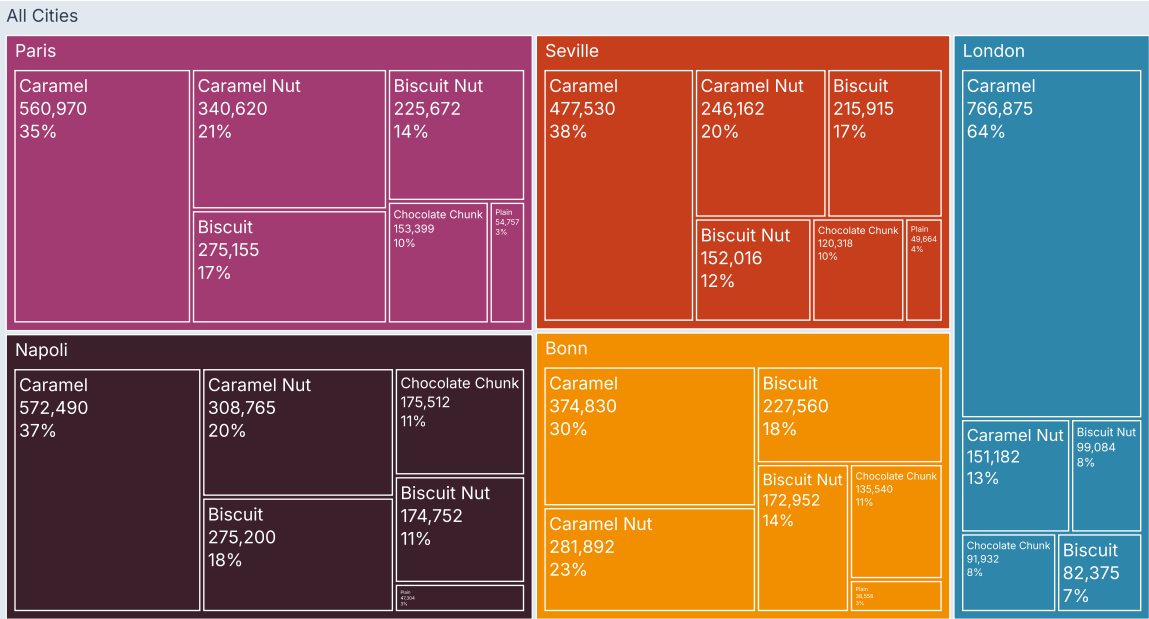




Revenue by Confectionery per City (Spider / Radar)



Revenue Breakdown: City → Confectionery (Treemap)



Units Sold vs Profit — by City & Confectionery

